



清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

严谨 · 勤奋 · 求实 · 创新

数字系统设计

05-计算核与数据流

贾弘洋 & 刘勇攀

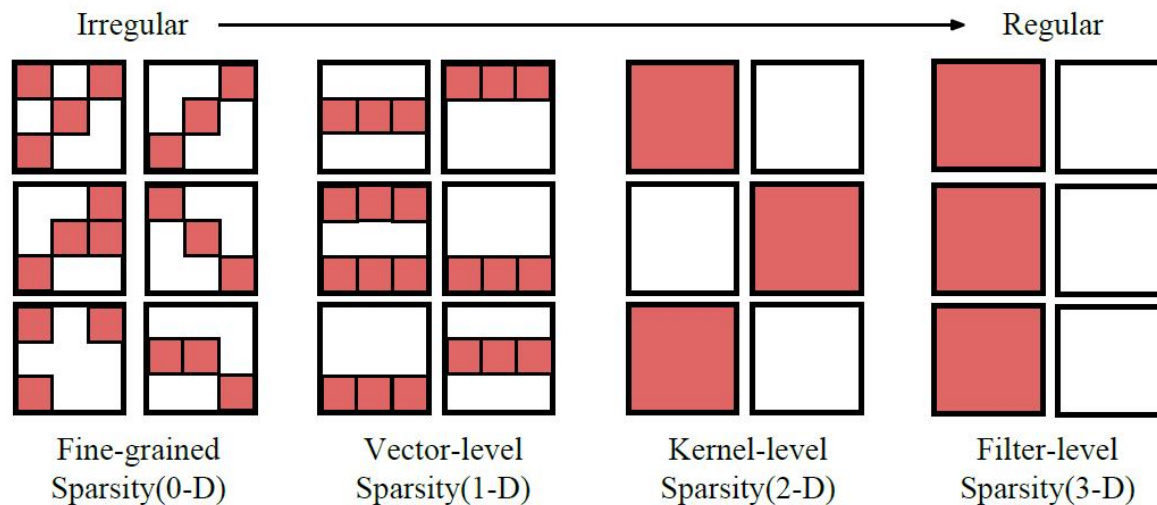
2025/03/18



/Review: sparsity for weights



- Unstructured and Structured Sparsity



Compression ratio

Sparsity granularity

Source: CVPR'17 Workshop

/Review: sparsity for weights

- Non-structured pruning
 - **Pros:** Higher compression rate
 - **Cons:** Only beneficial on dedicated devices
- Structured pruning:
 - **Pros:** Better suitable for more hardware devices
 - **Cons:** Lower compression rate

Could structure and non-structure pruning have similar compression ratio?

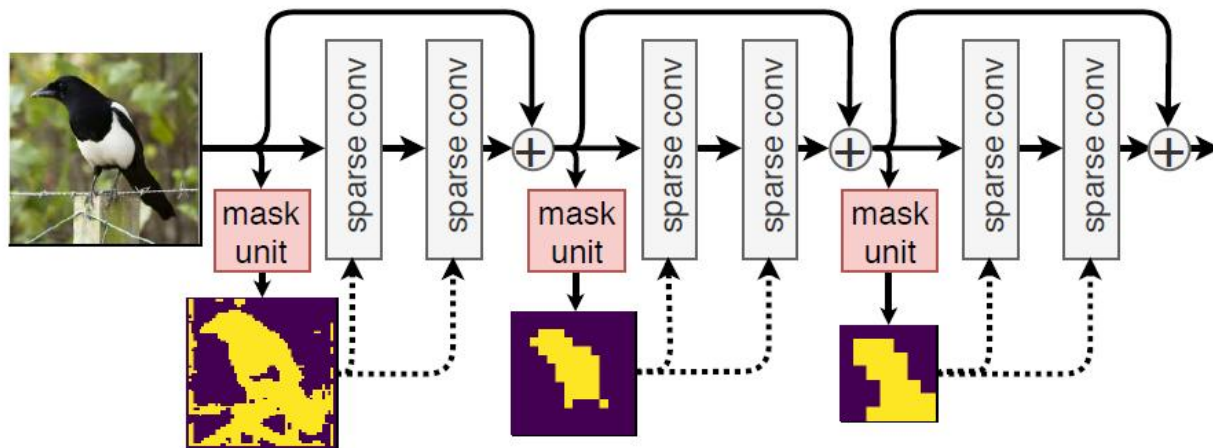
	GPU	Multi-core CPU	FPGA	ASIC
Non-Structured Pruning	Not Suitable	Not Suitable	Storage/Computation Dependent ?	
Structured Pruning	Suitable	Suitable		

Is non-structured pruning beneficial at all?

/Review: sparsity for activations

Thomas@CVPR,
2020

- Spatial-wise Sparsity



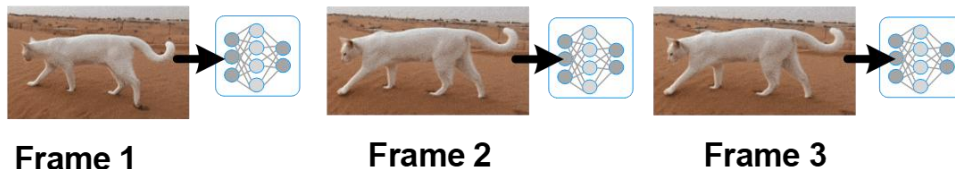
/Review: sparsity for activations



Yuan Z@ISSCC,
2020

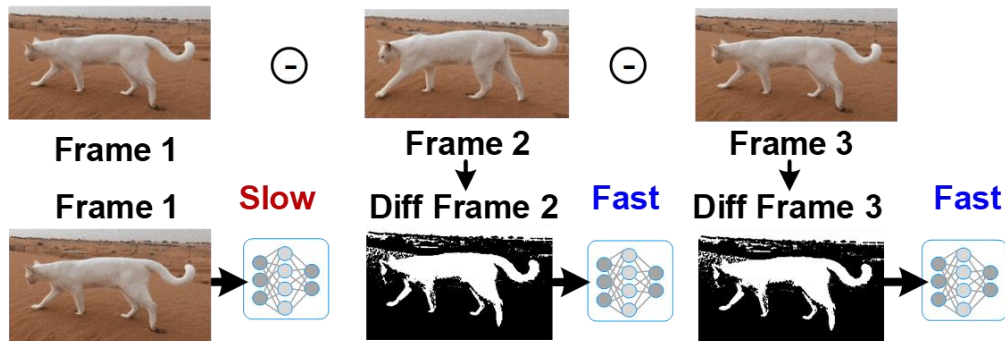
- Temporal-wise Sparsity

Traditional Way Process One by One



New Dimension Inter Frame Data Reuse

- Faster Processing Time
- Smaller Transmission Data
- Remain Some Challenges



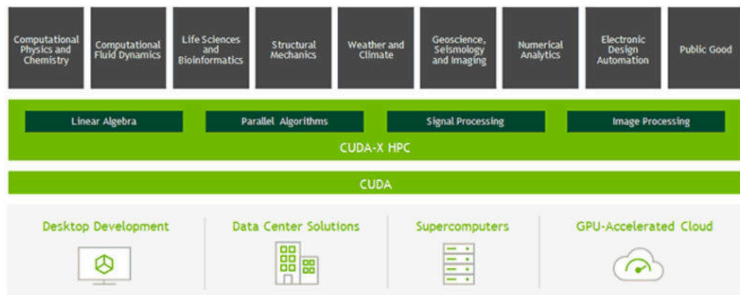
/Two Stories

BrookGPU: Introduction



As the programmability and performance of modern GPUs continues to increase, many researchers are looking to graphics hardware to solve problems previously performed on general purpose CPUs. In many cases, performing general purpose computation on graphics hardware can provide a significant advantage over implementations on traditional CPUs. However, if GPUs are to become a powerful processing resource, it is important to establish the correct abstraction of the hardware; this will encourage efficient application design as well as an optimizable interface for hardware designers.

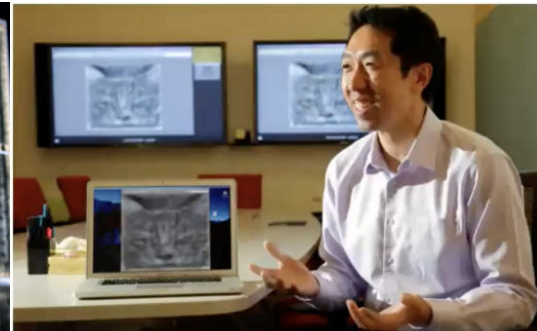
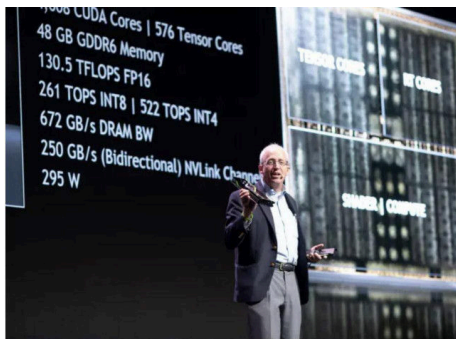
CUDA (Compute Unified Device Architecture)



How Many Computers to Identify a Cat? 16,000

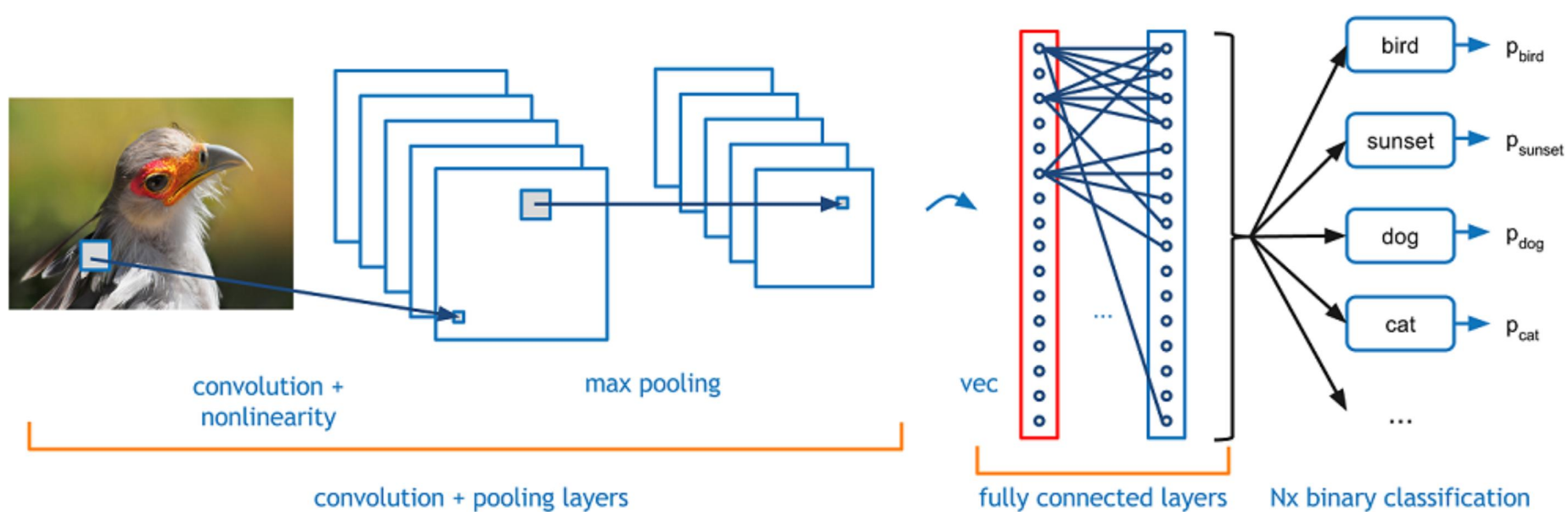
“Presented with 10 million digital images found in YouTube videos, what did Google’s brain do? What millions of humans do with YouTube: looked for cats.”

“Nvidia’s AI journey started at Joanie’s Cafe in Palo Alto, California, in 2010. “We got into AI after a breakfast meeting I had with Andrew Ng,” says Dally. At that breakfast, Ng, a well known AI researcher who was working with Google Brain at the time, explained how Google was training AI systems to recognise photos of cats with the help of 16,000 central processing units, or CPUs. After pointing out next to no one has 16,000 CPUs at their disposal, Dally laid down a challenge. “I bet we could do this with way fewer GPUs,” he said to Ng.”



/Outline

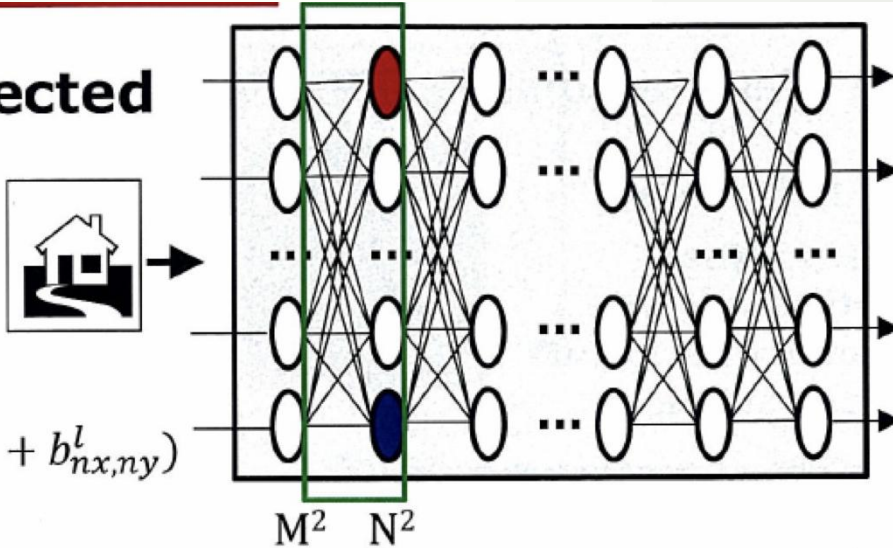
- **Convolution**
- **Transformation for convolution**
- **Pooling/BatchNorm**
- **Computing Flow**



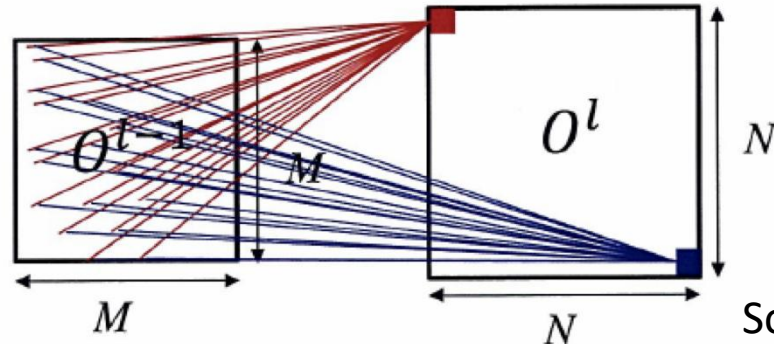
Fully Connected NN

DNN or fully connected
(Deep Neural Network)

$$o_{nx,ny}^l = \sigma \left(\sum_{mx,my}^{M,M} w_{mx,my,nx,ny}^l \cdot o_{mx,my}^{l-1} + b_{nx,ny}^l \right)$$



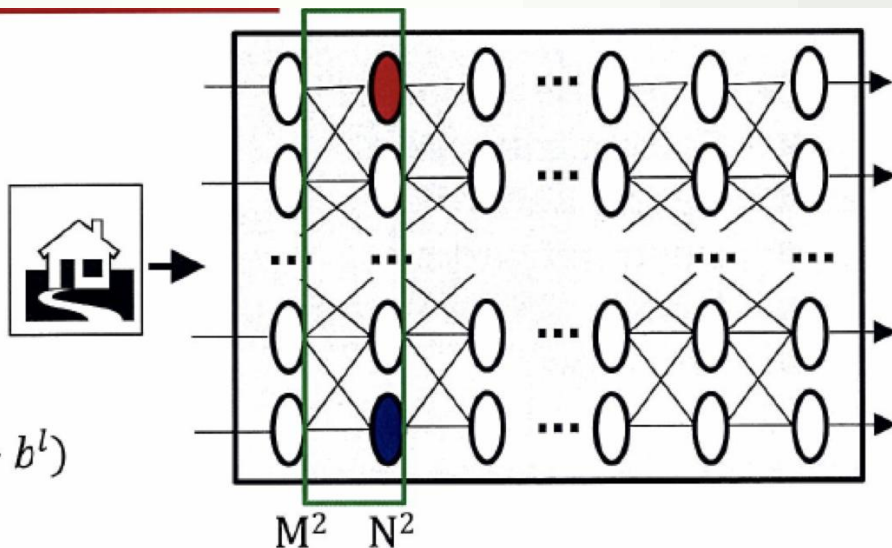
- M and N large!
- $M^2 \times N^2$ weights/layer
- Very flexible
- Memory ☹️, training ☹️



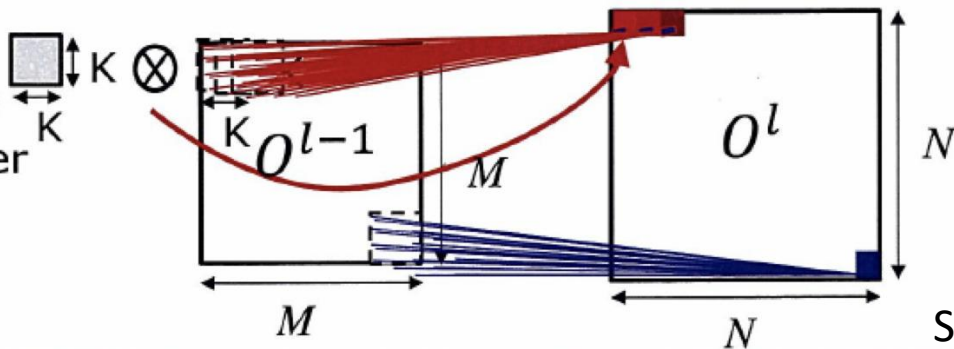
Convolutional NN

CNN or ConvNet (Convolutional Neural Network)

$$o_{nx,ny}^l = \sigma \left(\sum_{kx,ky}^{K,K} w_{kx,ky}^l \cdot o_{xn+kx,ny+ky}^{l-1} + b^l \right)$$



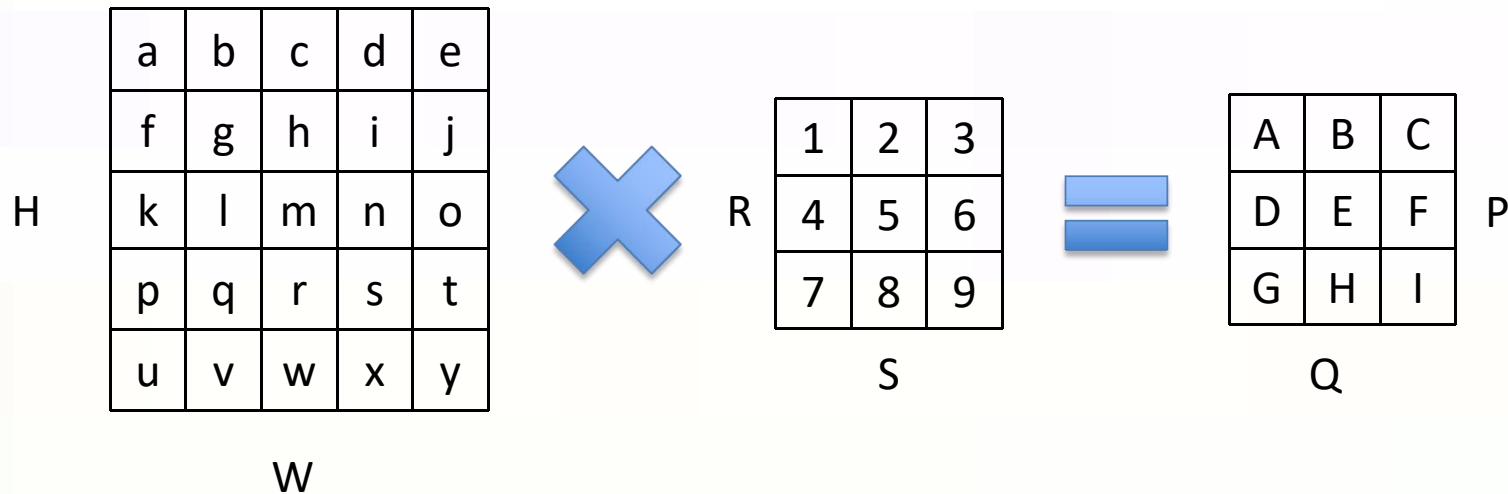
- K^2 weights/layer
- K small ($1 \leq K \leq 11$)
- Less operations per output
- Significantly less weights!



/Convolution

- **Basic 2-D convolution**
- **3-D convolution**
- **Depth-wise convolution & group convolution**
- **Misc convolution**

/ 2D Convolution



H: height of input activation

W: width of input activation

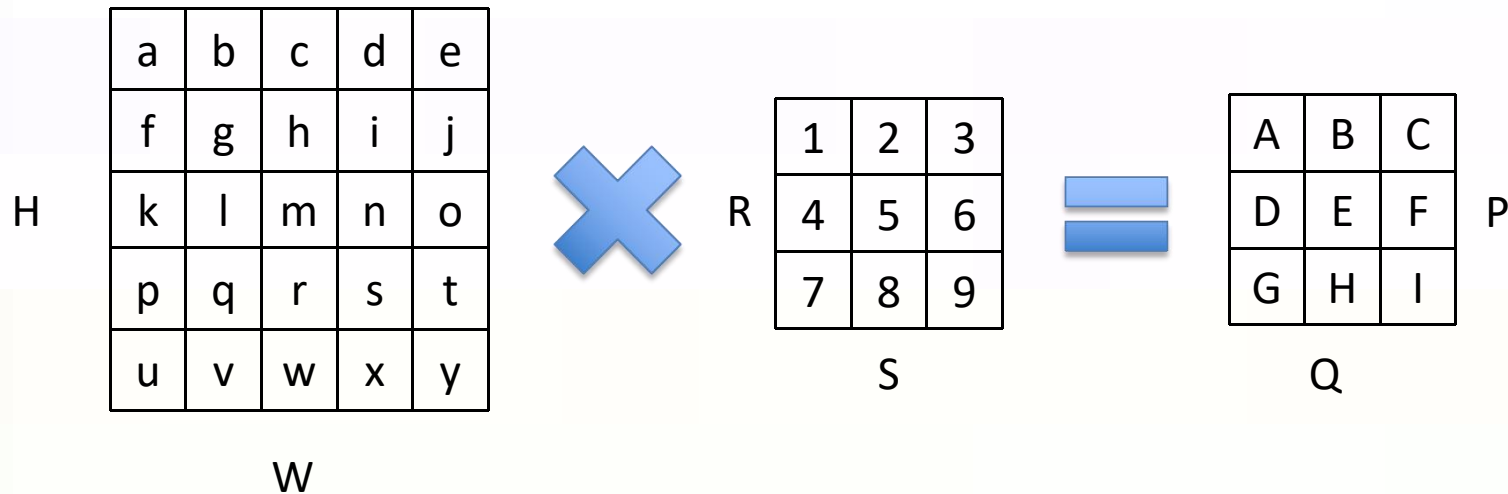
R: height of weight

S: width of weight

P: height of output activation

Q: width of output activation

/ 2D Convolution



$$P = \frac{H - R}{stride} + 1$$

$$Q = \frac{W - S}{stride} + 1$$

Stride: # of rows/columns traversed per step

/ 2D Convolution

H

0	0	0	0	0	0	0
0	a	b	c	d	e	0
0	f	g	h	i	j	0
0	k	l	m	n	o	0
0	p	q	r	s	t	0
0	u	v	w	x	y	0
0	0	0	0	0	0	0

W



加入padding, 使输入输出空间尺度不变

R

1	2	3
4	5	6
7	8	9

S



A	B	C	D	E
F	G	H	I	J
K	L	M	N	O
P	Q	R	S	T
U	V	W	X	Y

P

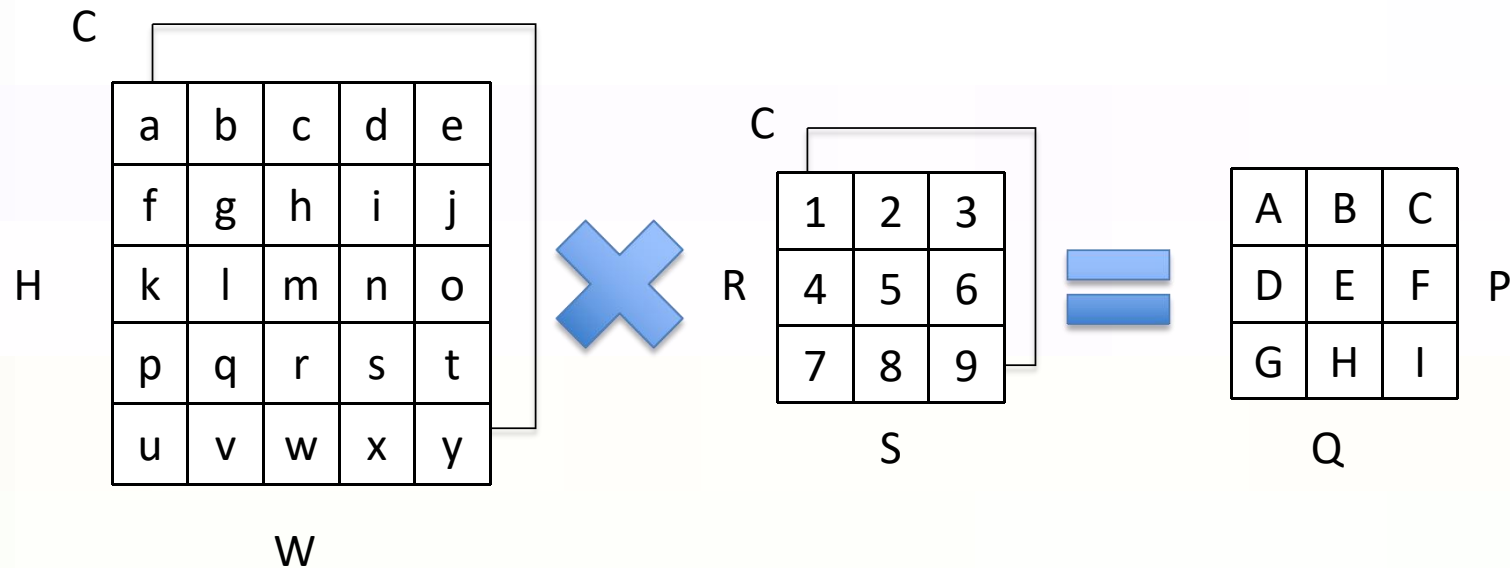
Q

$$P = \frac{H - R + 2 * pad}{stride} + 1$$

$$Q = \frac{W - S + 2 * pad}{stride} + 1$$

padding: # of zero rows/columns added

/ 3D Convolution



H: height of input activation

W: width of input activation

C: # of input channels

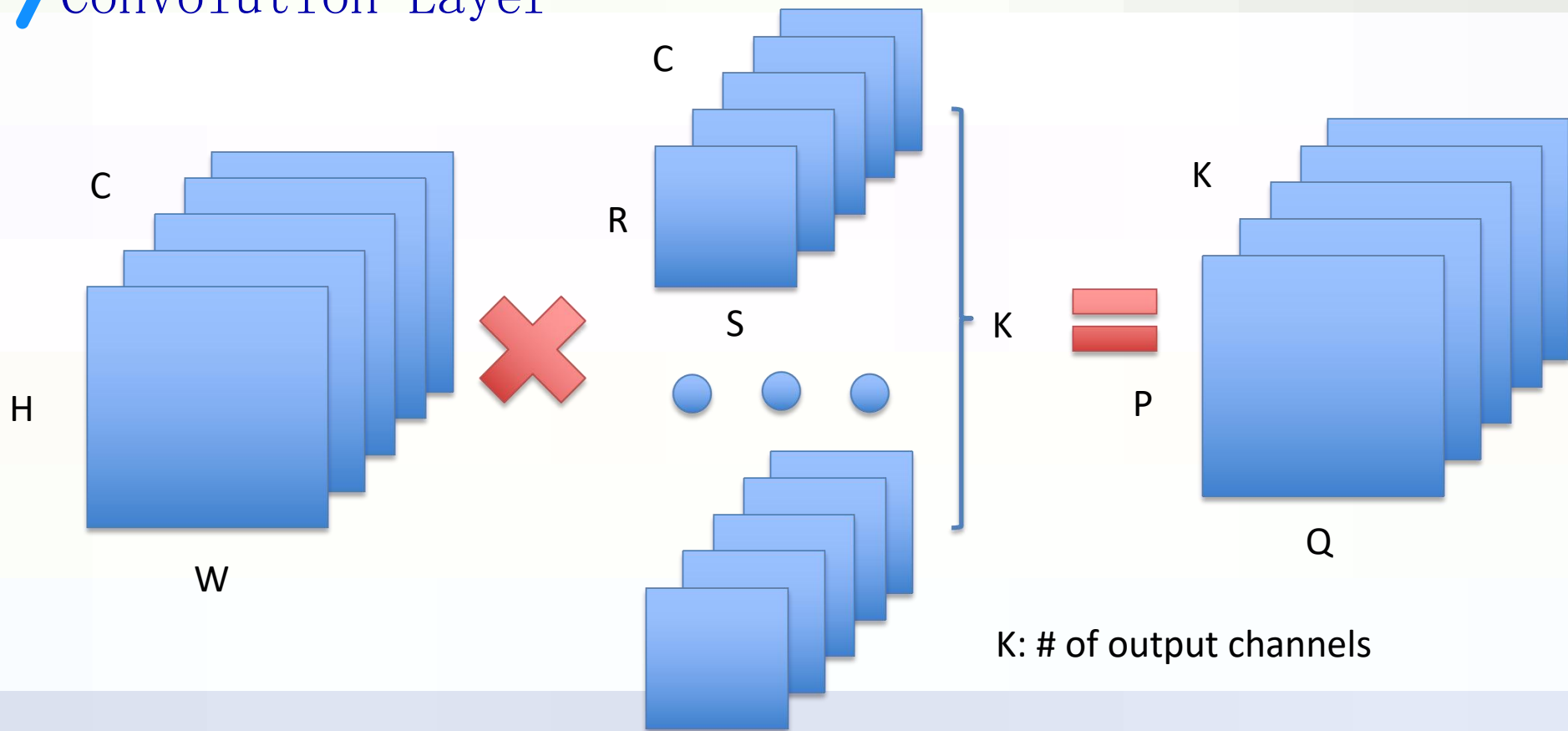
R: height of weight

S: width of weight

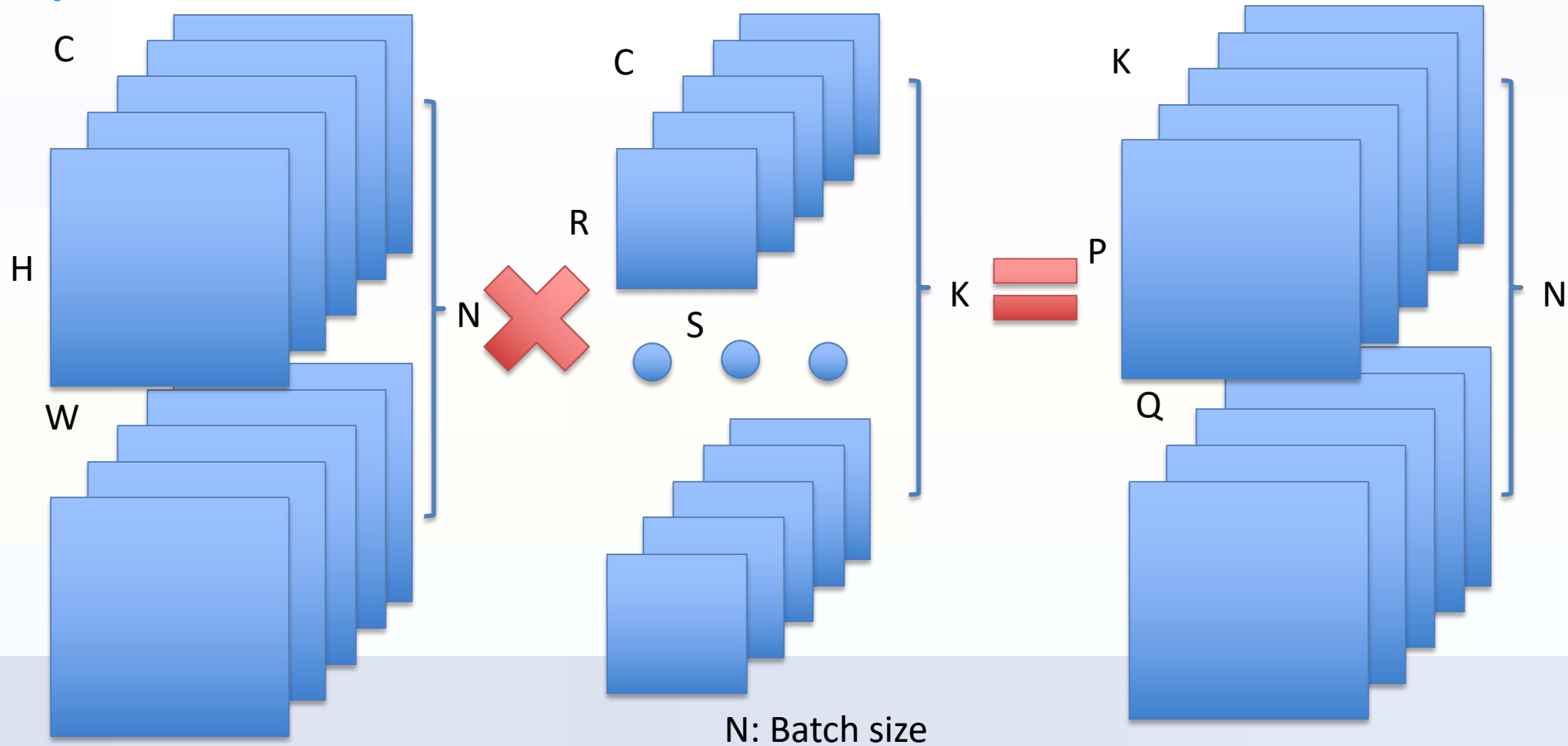
P: height of output activation

Q: width of output activation

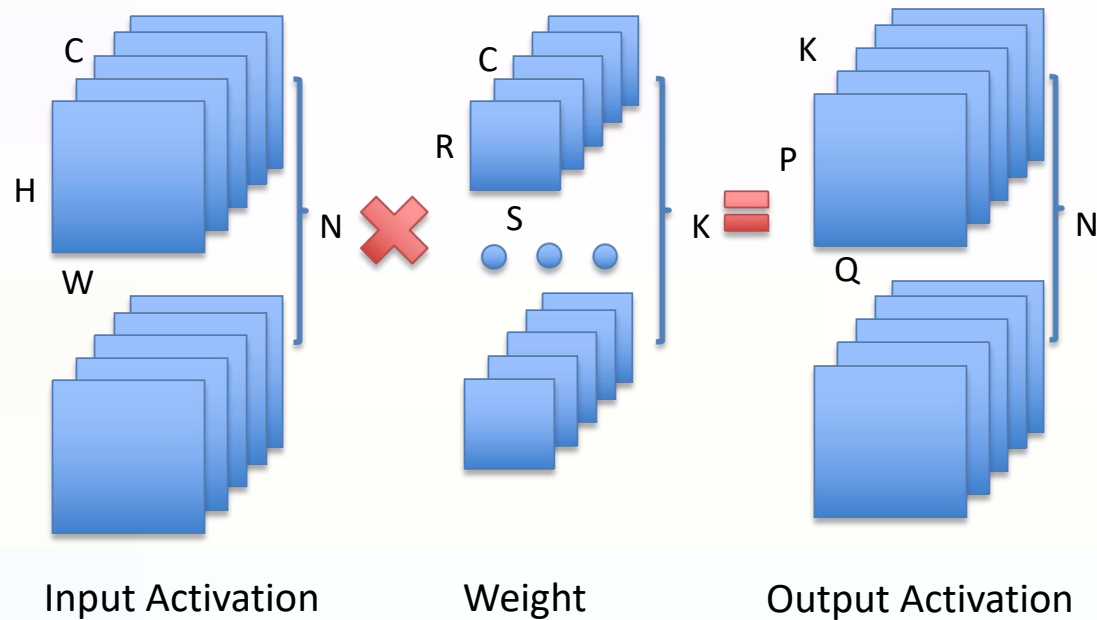
/Convolution Layer



/Convolution Layer (Batched)



/Convolution Layer



H: height of input activation

W: width of input activation

R: height of weight

S: width of weight

P: height of output activation

Q: width of output activation

stride: # of rows/columns traversed per step

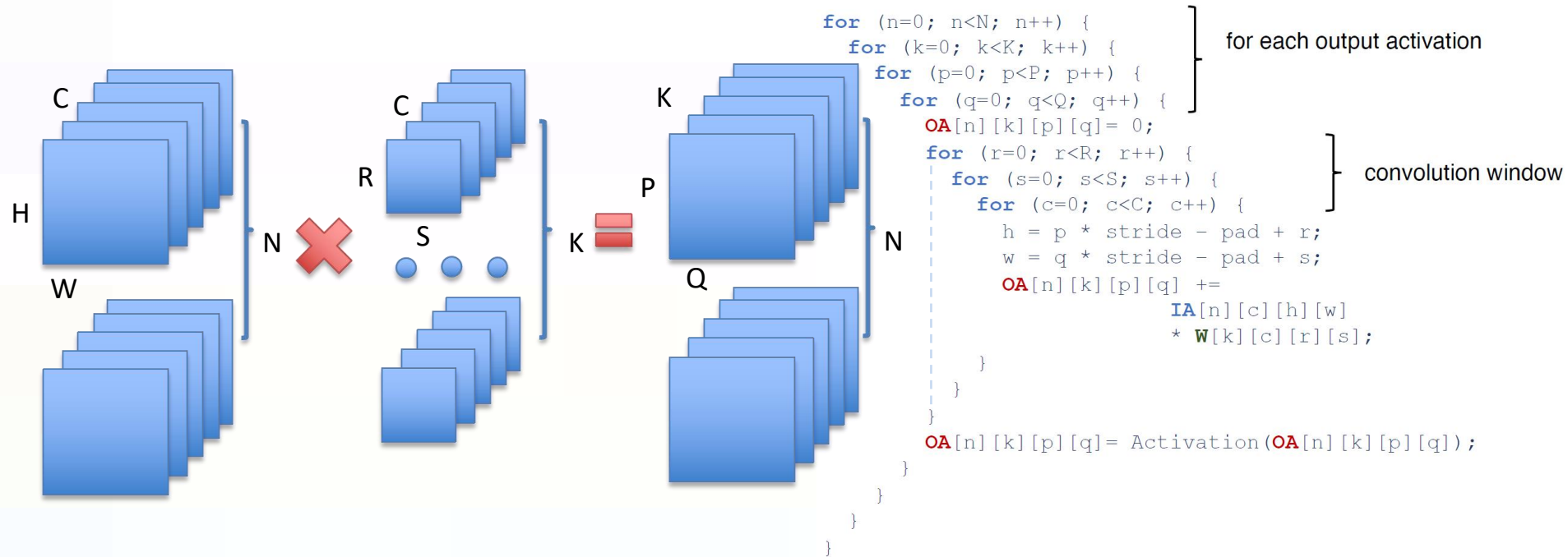
padding: # of zero rows/columns added

C: # of input channels

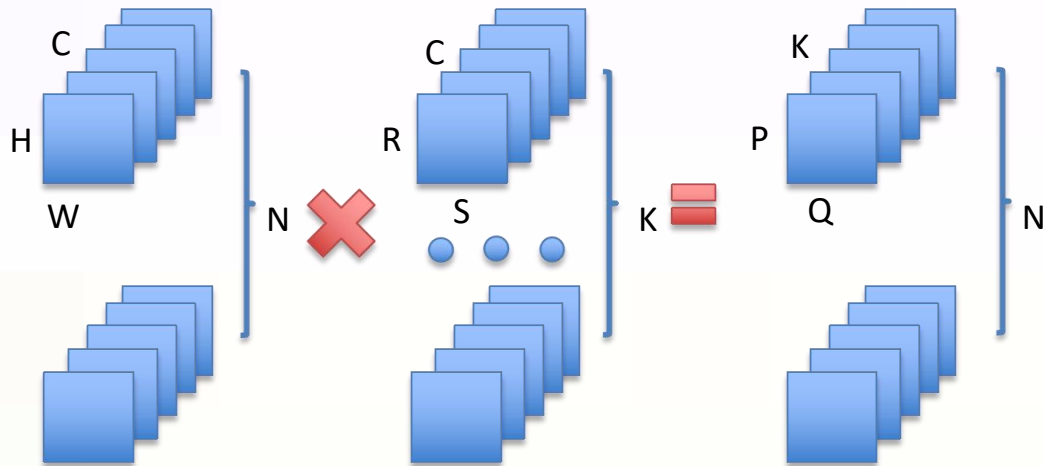
K: # of output channels

N: Batch size

/Convolution Layer



/Fully-Connected Layer



$H = 1$

$W = 1$

$R = 1$

$S = 1$

$P = 1$

$Q = 1$

Stride = 1

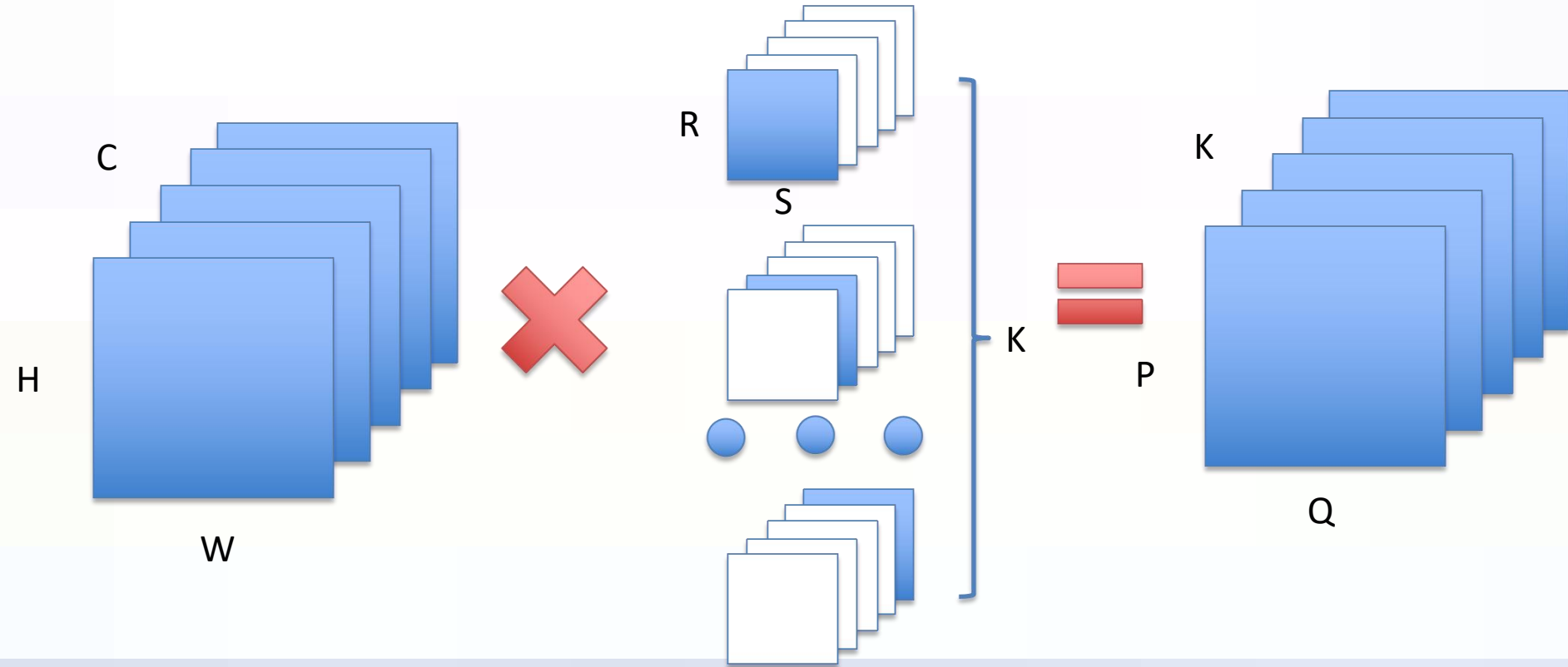
Padding = 0

C: # of input channels

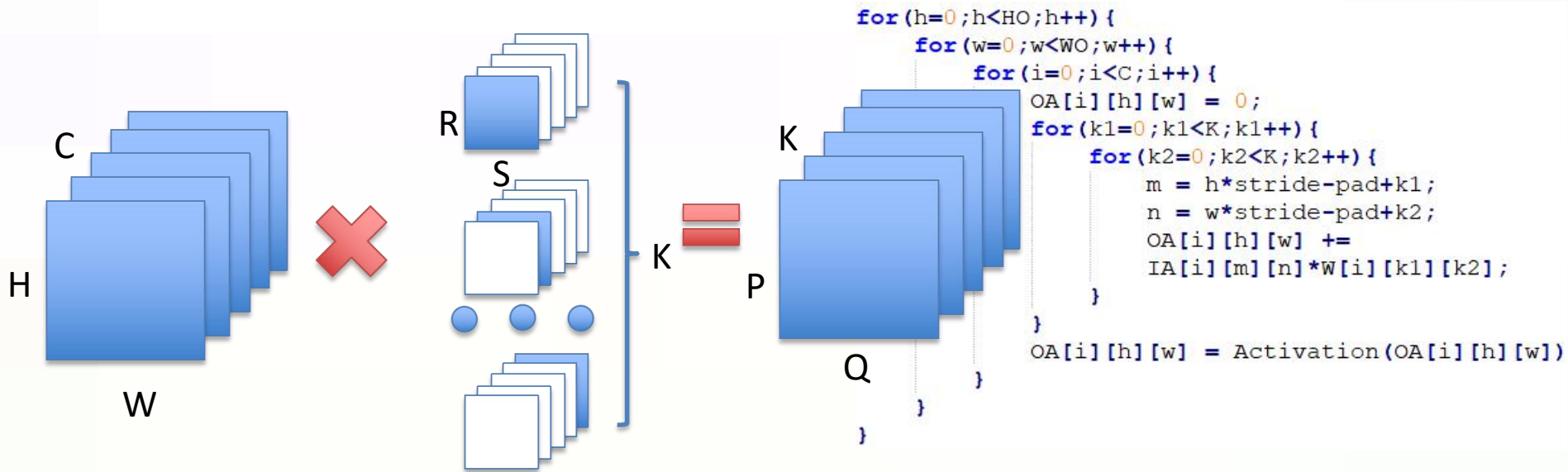
K: # of output channels

N: Batch size

/Depth-wise Convolution Layer

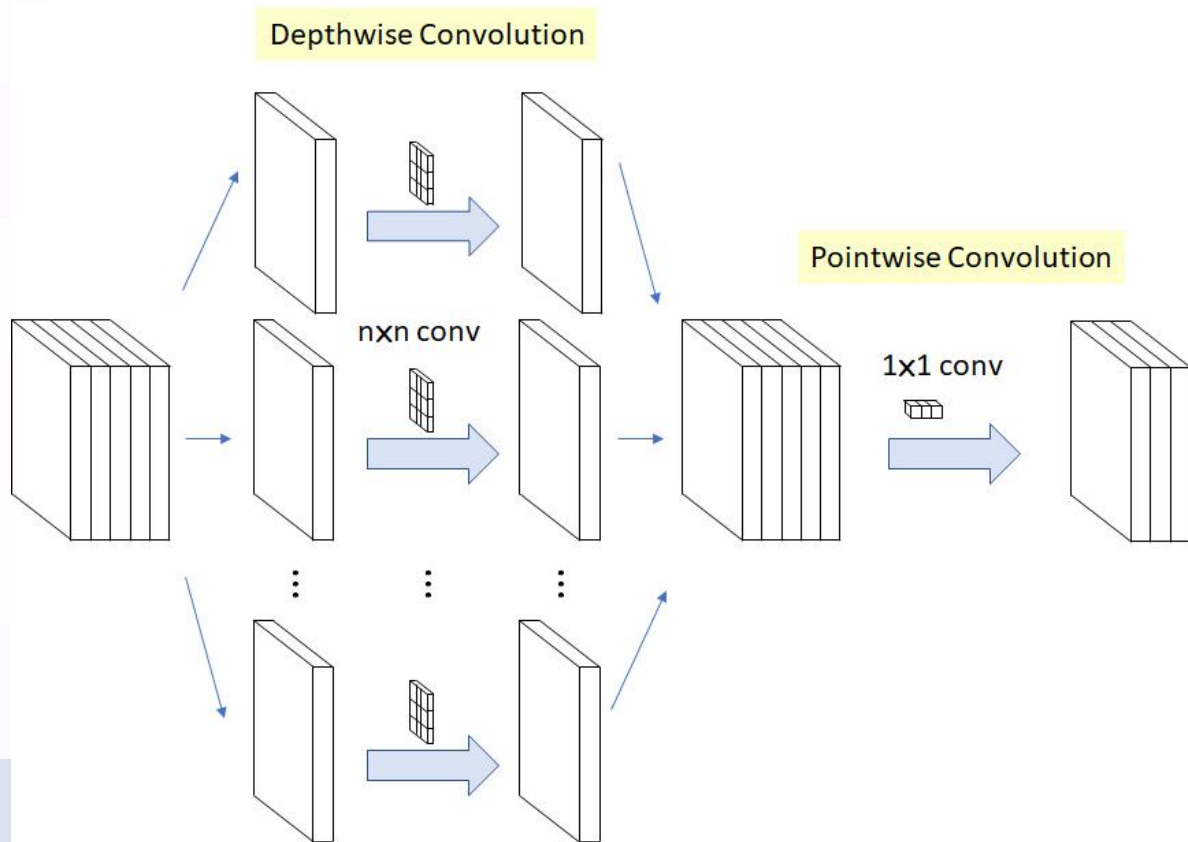


/Depth-wise Convolution Layer



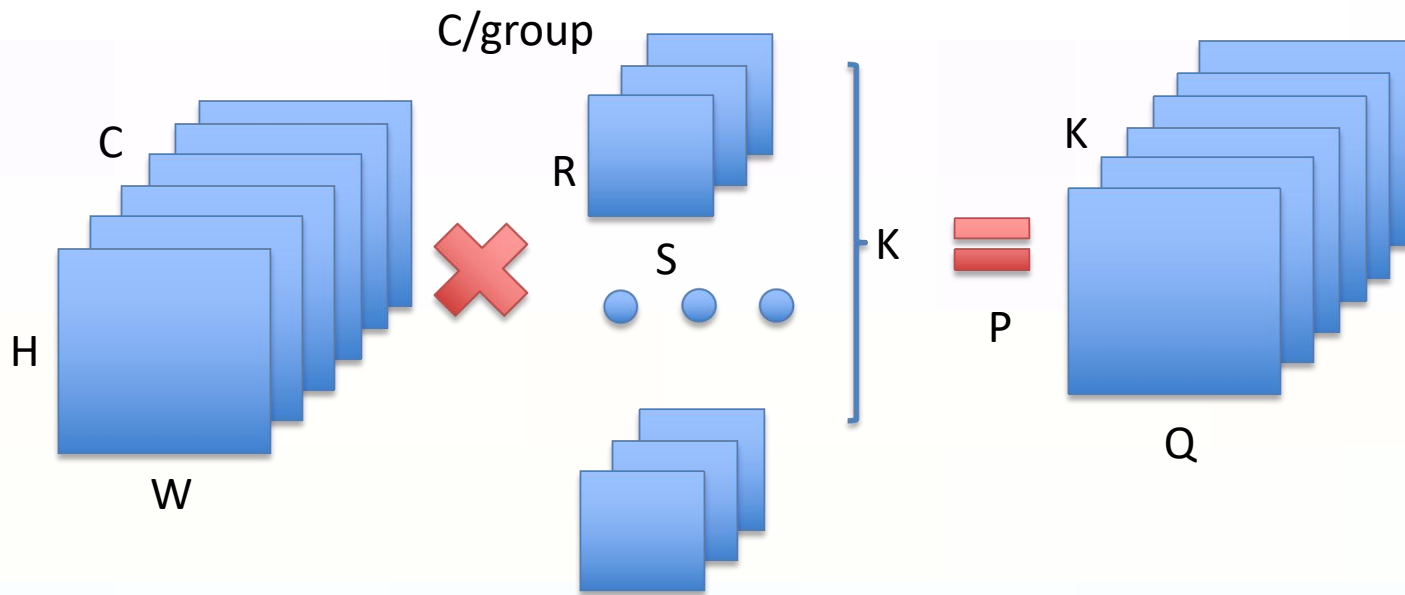
通常后续会接 1×1 的普通卷积层，又称point-wise convolution

/Depth-wise Convolution Layer



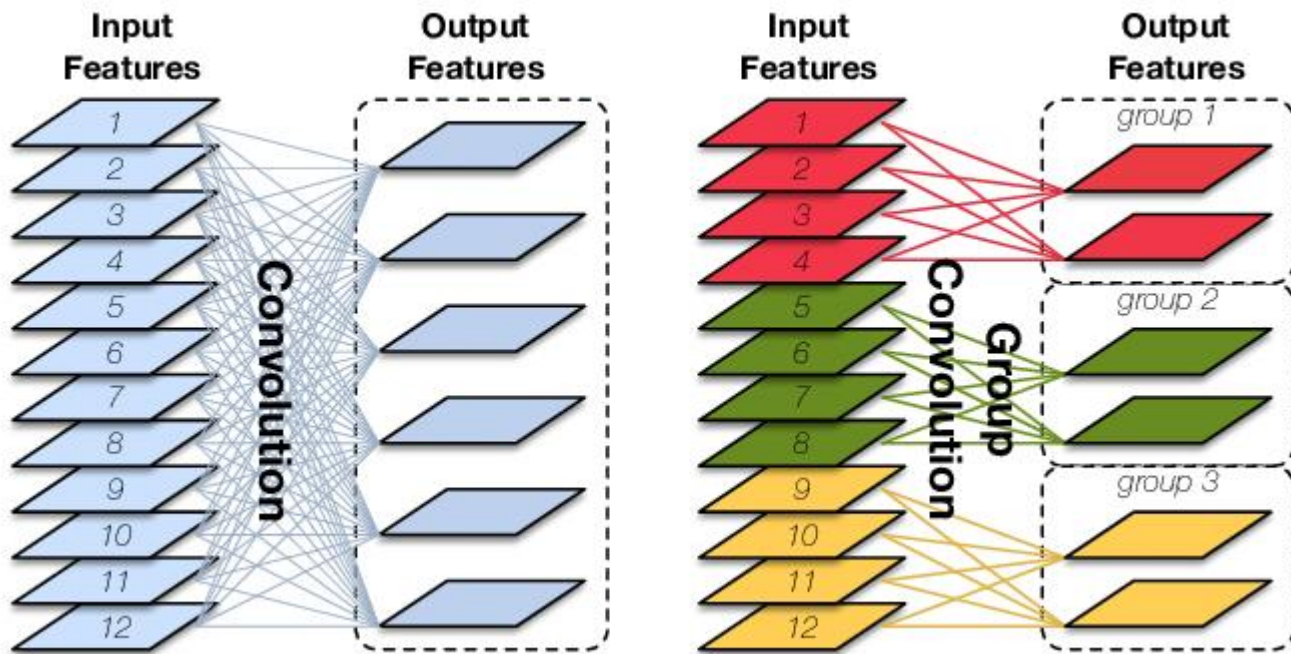
常用于轻量级网络的设计中，例如
MobileNet系列

/Group Convolution



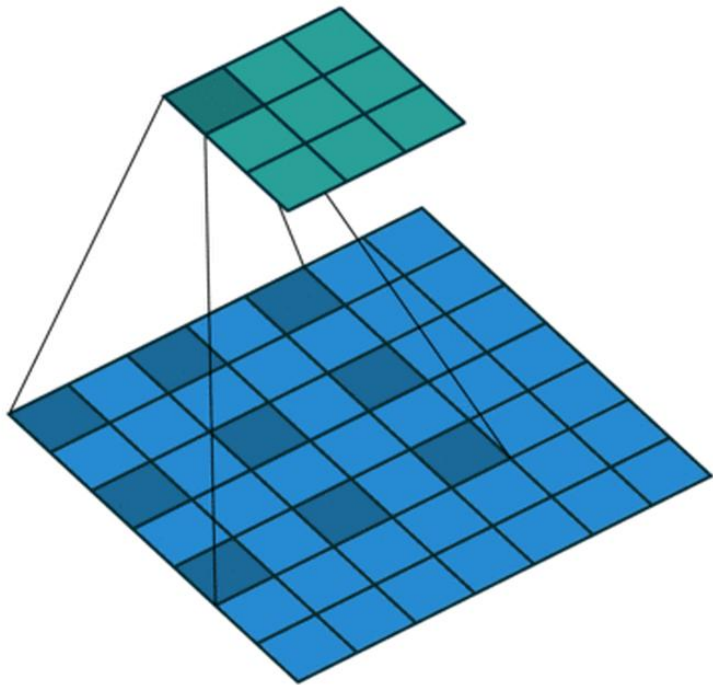
Depth-wise convolution是group=C的特殊情形

/ Group Convolution



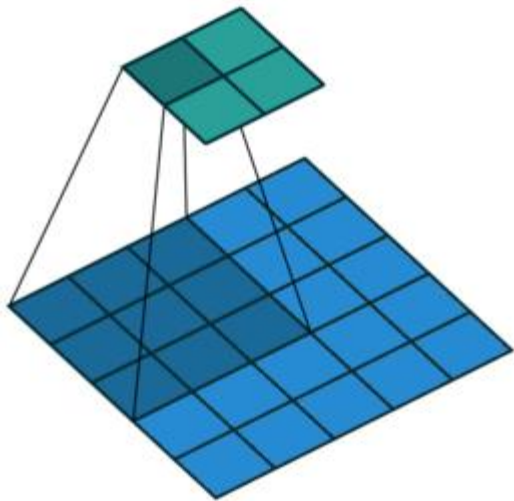
Depth-wise convolution是group=C的特殊情形

/Dilated Convolution

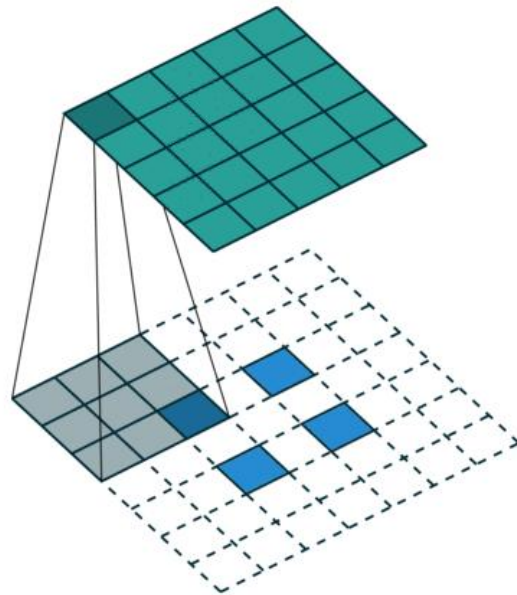


- Basic convolution is the convolution with $\text{dilated}=1$
- Need larger padding if we want to remain the spatial size.

/ Deconvolution



Convolution with stride=2



Deconvolution with stride=2

$$H_{\text{out}} = (H_{\text{in}} - 1) * \text{stride} - 2 * \text{padding} + K$$

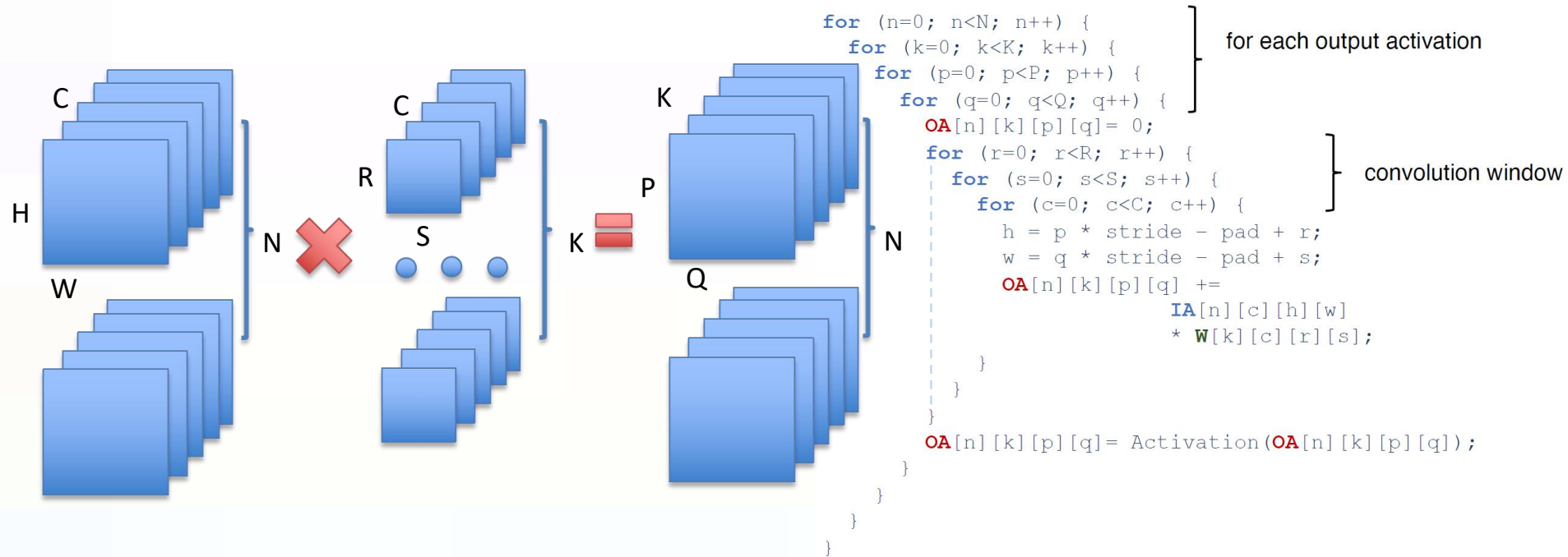
/Outline

- **Convolution**
- **Transformation for convolution**
- **Pooling/BatchNorm**
- **Computing Flow**

/Transformation for Convolution

- **Direct convolution**
- **GEMM**
- **FFT-based convolution**
- **Winograd convolution**

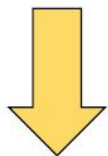
/Direct Convolution



- **Motivation:** 将卷积操作转换为矩阵乘操作
 - 利用成熟的矩阵运算加速手段
 - 是Caffe等工具采取的神经网络加速手段



Convolution



GEMM

Weight

R	1	2
	3	4

S



Input Activation

H

a	b	c
d	e	f
h	i	j

W



Output Activation

P

A	B
C	D

Q



1	2	3	4
---	---	---	---



a	b	d	e
b	c	e	f
d	e	h	i
e	f	i	j



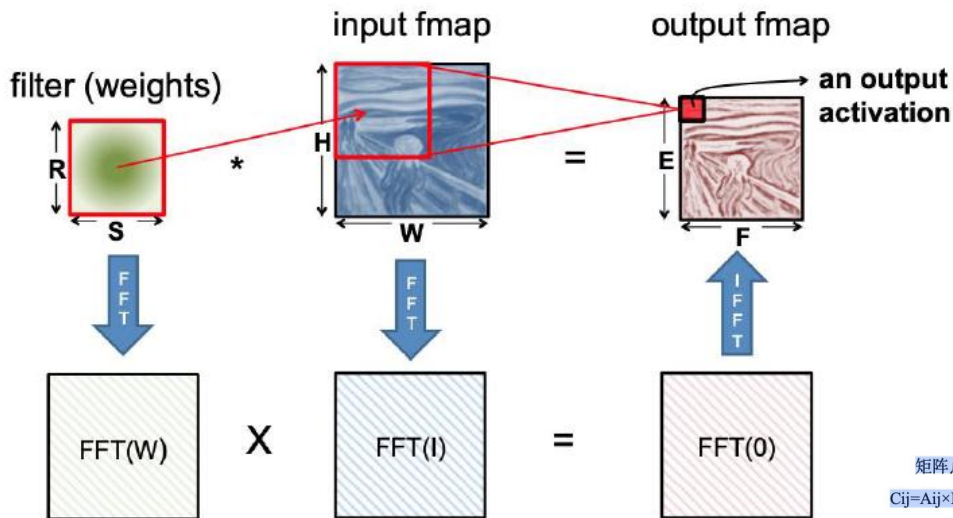
A	B	C	D
---	---	---	---

FFT-based convolution

- Motivation: 时域卷积等于频域相乘

$$f * g = \mathcal{F}^{-1} \{ \mathcal{F}\{f\} \cdot \mathcal{F}\{g\} \}$$

$\mathcal{F}\{f\}$ and $\mathcal{F}\{g\}$ are the Fourier transforms of f and g
The asterisk denotes convolution, not multiplication.



矩阵点乘计算公式：矩阵 A 与 B 点乘得到矩阵 C 的公式：
 $C_{ij} = A_{ij} \times B_{ij}$ 。其中 i 和 j 分别表示矩阵 A 与 B 的行数和列数。

Winograd convolution

- Motivation: 通过变换减少乘法次数
- Example: $IA=[d_0, d_1, d_2, d_3]$, $conv=[g_0, g_1, g_2]$, 1-D convolution. After $img2col$, we can have

$$OA = \begin{bmatrix} d_0 & d_1 & d_2 \\ d_1 & d_2 & d_3 \end{bmatrix} \begin{bmatrix} g_0 \\ g_1 \\ g_2 \end{bmatrix}$$

Before: 需要六次乘法，四次加法

For Winograd, suppose

$$\begin{aligned} m_1 &= (d_0 - d_2)g_0 & m_2 &= (d_1 + d_2)\frac{g_0 + g_1 + g_2}{2} \\ m_4 &= (d_1 - d_3)g_2 & m_3 &= (d_2 - d_1)\frac{g_0 - g_1 + g_2}{2} \end{aligned}$$

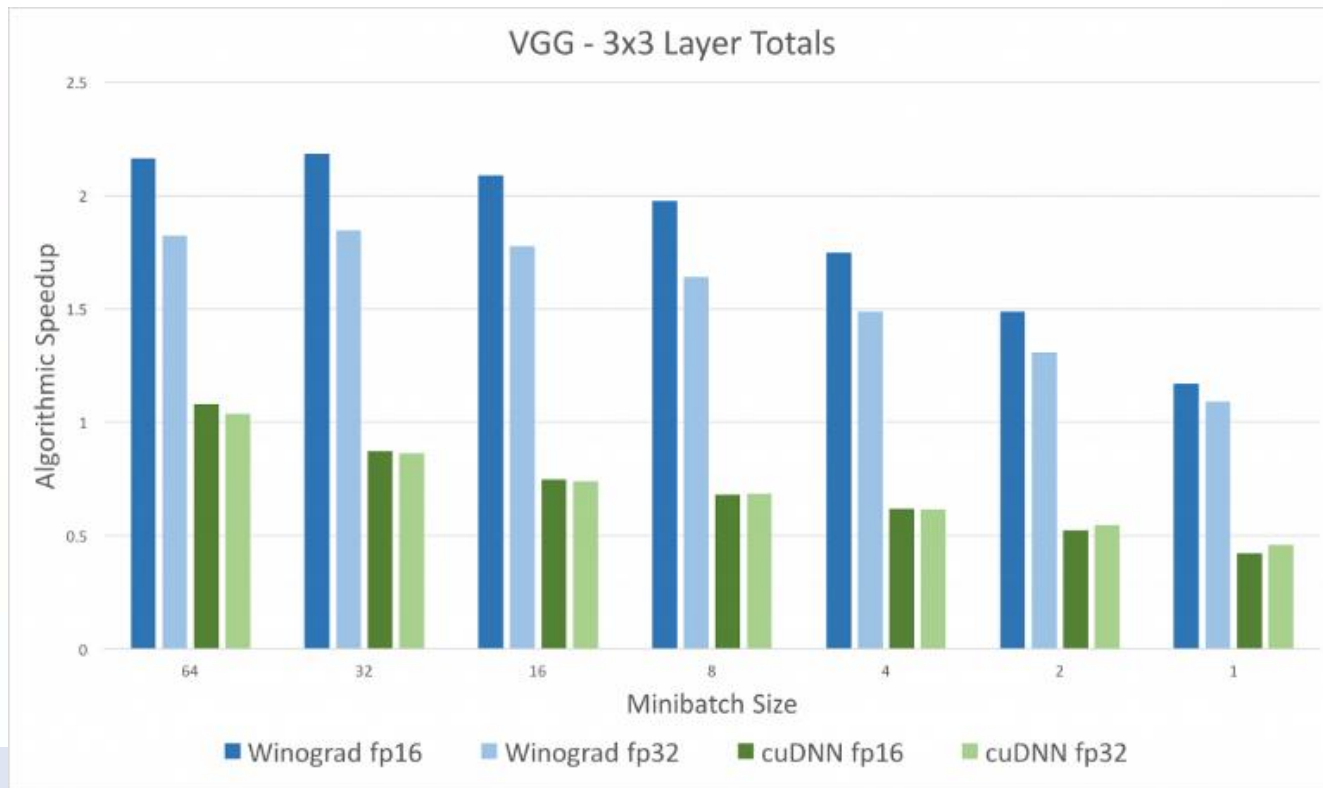
Then we can have

$$OA = \begin{bmatrix} m_1 + m_2 + m_3 \\ m_2 - m_3 - m_4 \end{bmatrix}$$

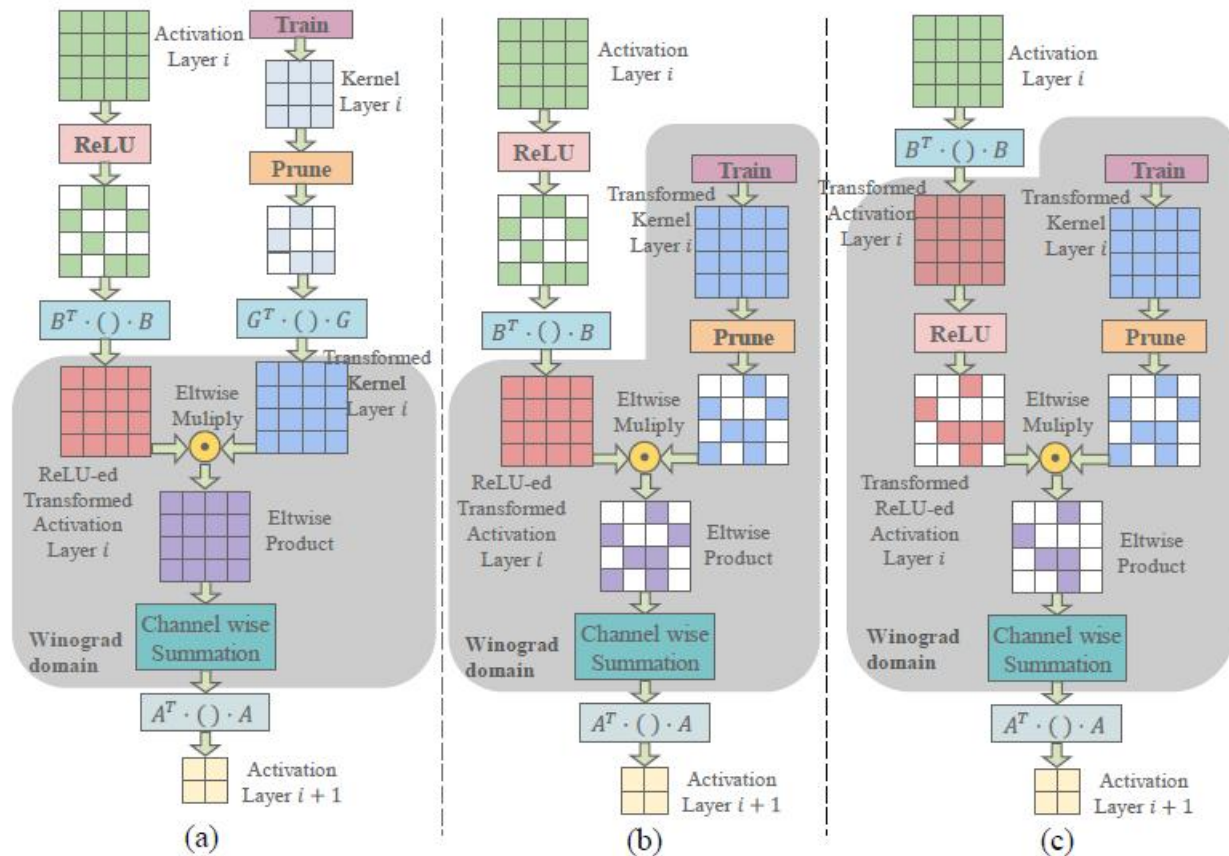
After: 需要四次乘法，八次加法

$(g_0+g_1+g_2)/2$ 和 $(g_0-g_1+g_2)/2$ 可以复用

/ Winograd convolution



Winograd convolution



ICLR, 2018

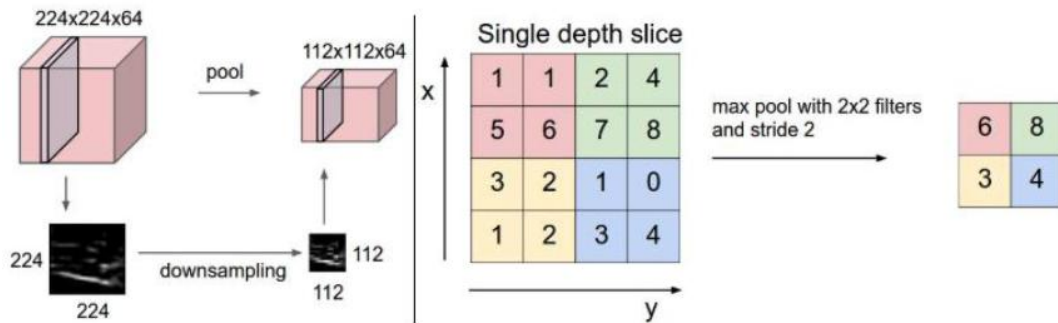
- Winograd卷积会破坏原有的稀疏性（如图(a)所示），这会使得传统的剪枝方法无效化。
- 图(b)，直接对winograd变换后的weight进行剪枝
- 图(c)，进一步修改结构，保留activation sparsity

/Outline

- **Convolution**
- **Transformation for convolution**
- **Pooling/BatchNorm**
- **Computing Flow**

/ Pooling

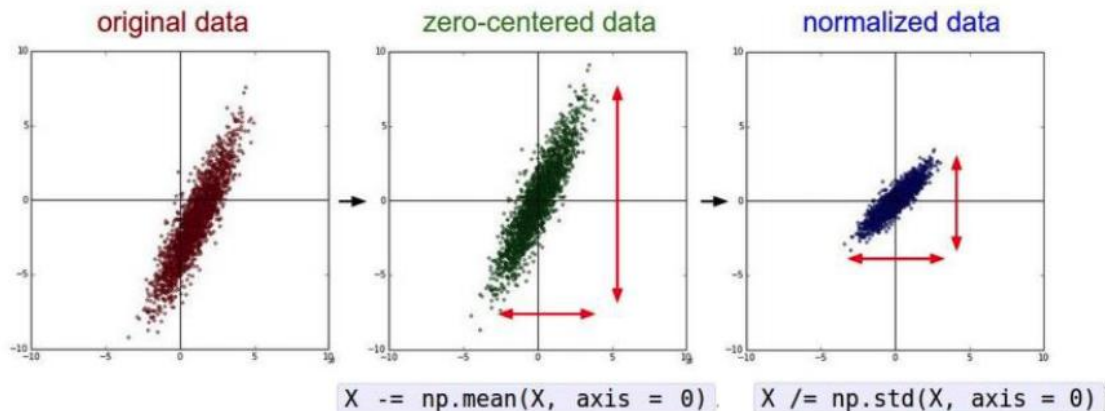
- Progressively reduce the spatial size of the representation to reduce the amount of parameters and computation in the network, and hence to also control overfitting.
- Parameters:
 - Type: MAX and AVG
 - Pooling kernel size
 - Pooling stride



Pooling layer downsamples the volume spatially, independently in each depth slice of the input volume. Left: In this example, the input volume of size $[224 \times 224 \times 64]$ is pooled with filter size 2, stride 2 into output volume of size $[112 \times 112 \times 64]$. Notice that the volume depth is preserved. Right: The most common downsampling operation is max, giving rise to max pooling, here shown with a stride of 2. That is, each max is taken over 4 numbers (little 2×2 square).

/Batchnorm

- Goal: make it easier to train by providing zero-mean, unit-variance activations.
 - The training is complicated by the fact that the inputs to each layer are affected by the parameters of all preceding layers – so that small changes to the network parameters amplify as the network becomes deeper.



Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift

/Batchnorm

Training Time

Input: $x : N \times D$

Learnable scale and shift parameters:

$$\gamma, \beta : D$$

Learning $\gamma = \sigma$,
 $\beta = \mu$, will recover the
identity function!

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$

Per-channel mean,
shape is D

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

Per-channel var,
shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Normalized x,
Shape is N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

Output,
Shape is N x D

Test Time

$$\mu_j = \text{(Running) average of values seen during training}$$

Per-channel mean,
shape is D

$$\sigma_j^2 = \text{(Running) average of values seen during training}$$

Per-channel var,
shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Normalized x,
Shape is N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

Output,
Shape is N x D

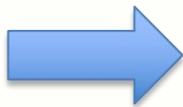
/Batchnorm

- Merge the Batchnorm into convolution during testing



$$y = w * x + b$$

$$z = \frac{y - \text{mean}}{\sqrt{\text{var}}} * \beta + \gamma$$



$$z = w' * x + b'$$

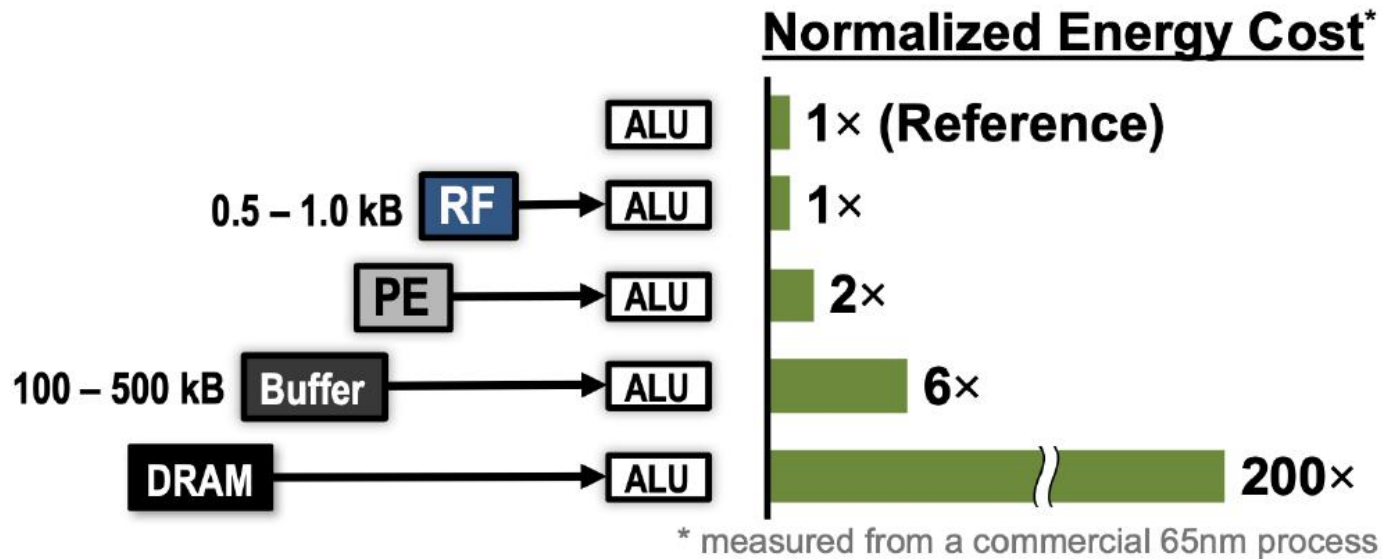
$$w' = \frac{w}{\sqrt{\text{var}}} * \beta$$

$$b' = \frac{b - \text{mean}}{\sqrt{\text{var}}} * \beta + \gamma$$

/Outline

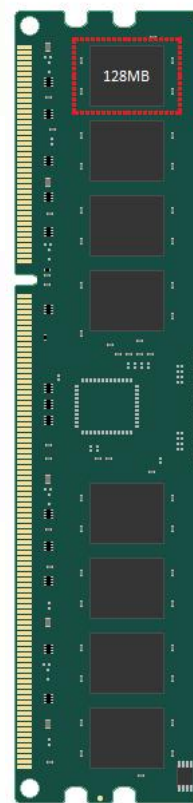
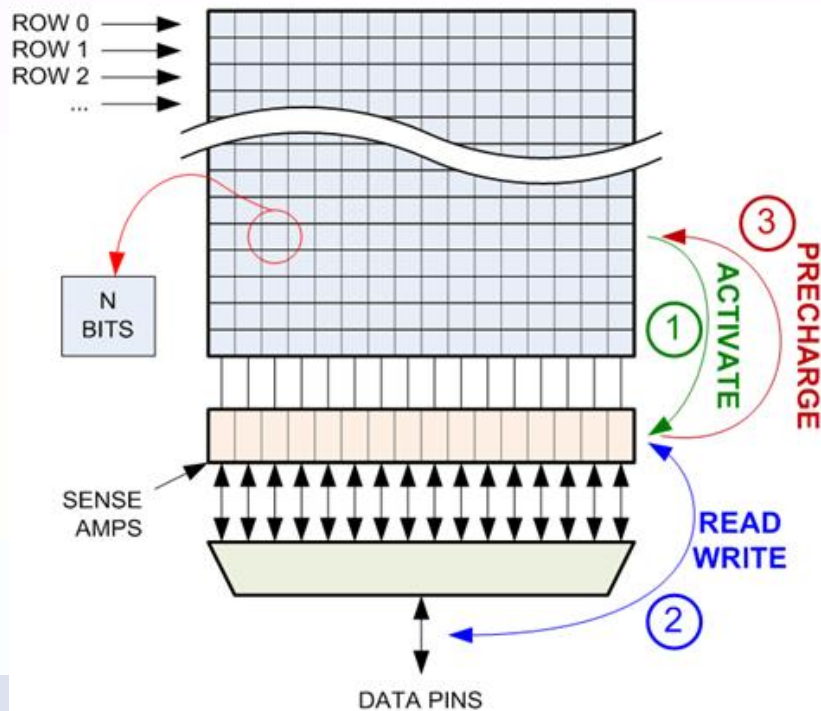
- **Convolution**
- **Transformation for convolution**
- **Pooling/BatchNorm**
- **Computing Flow**

/Principle-locality



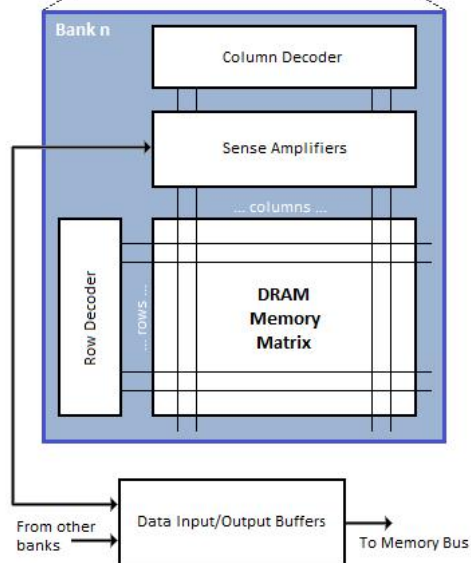
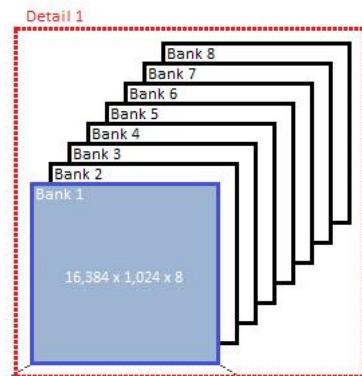
/Principle-locality

- DRAM

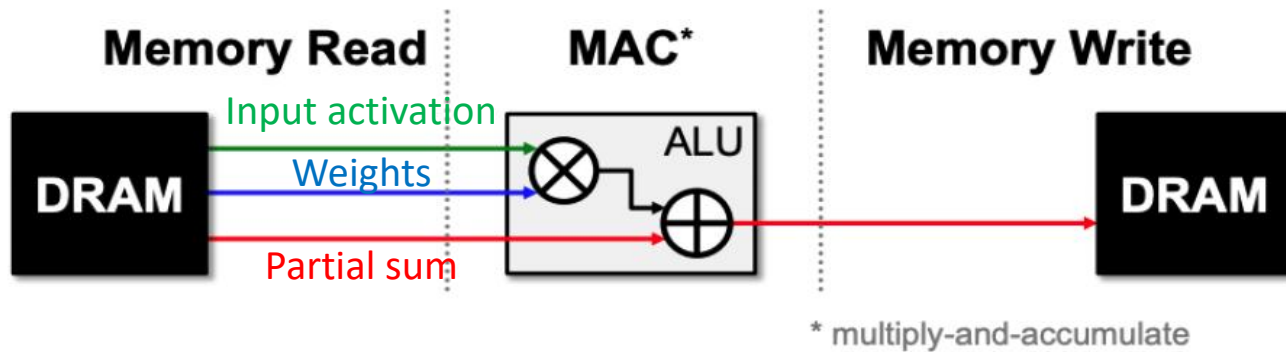


2GB DDR3 Dual Inline Memory Module (DIMM)

Rank 1 (side 1)



/Principle-locality



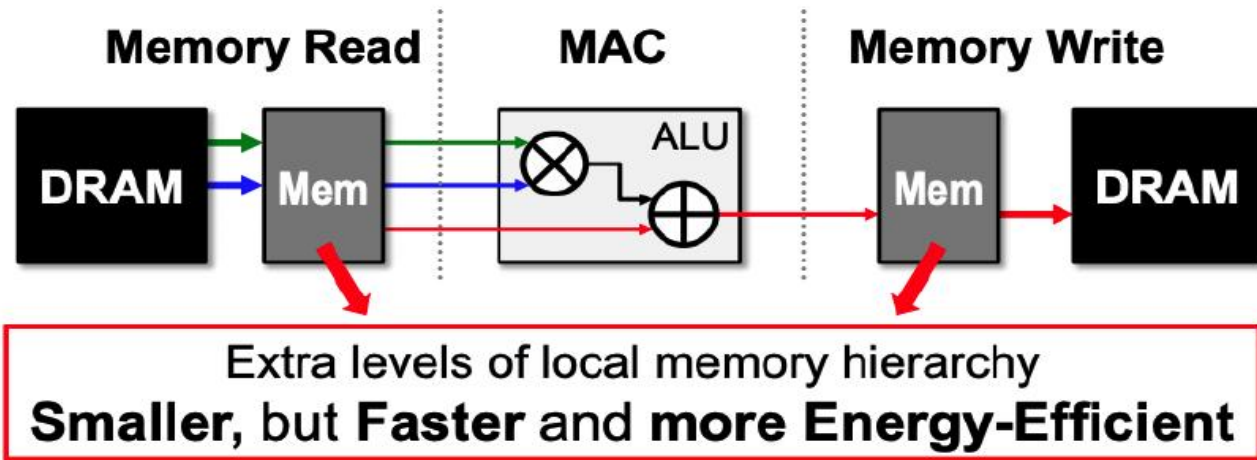
/ Principle-locality

- **Temporal** reuse: the same data is used more than once over time by the same consumer.
- **Spatial** reuse: the same data is used by **more than one consumer** at **different spatial locations** of the hardware.



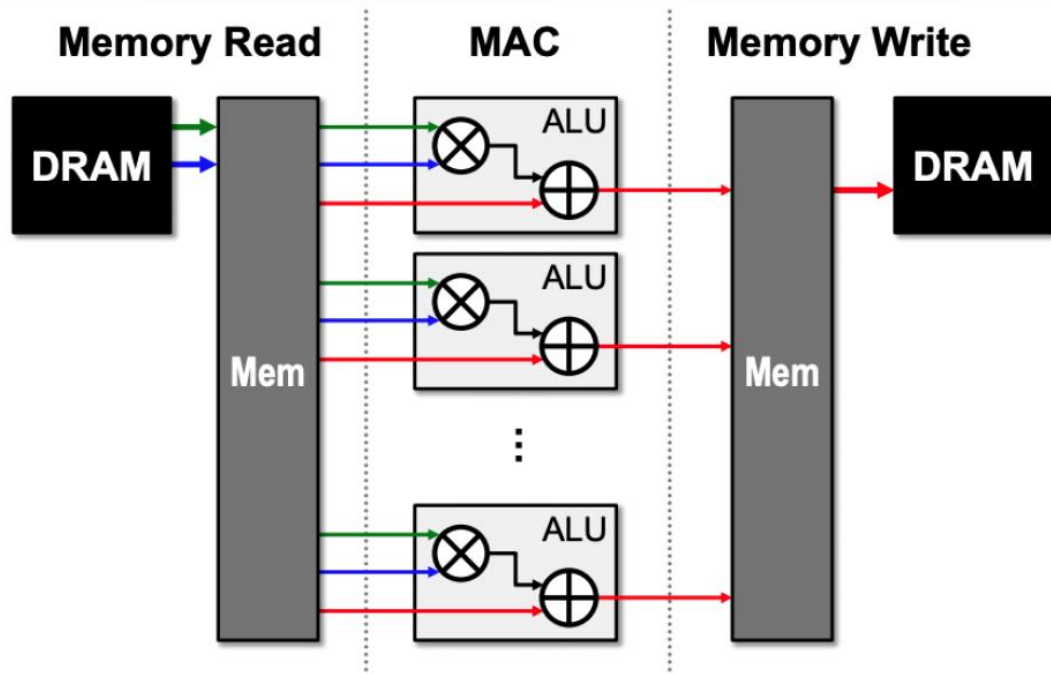
/Principle-locality

- **Temporal** reuse: the same data is used more than once over time by the same consumer.

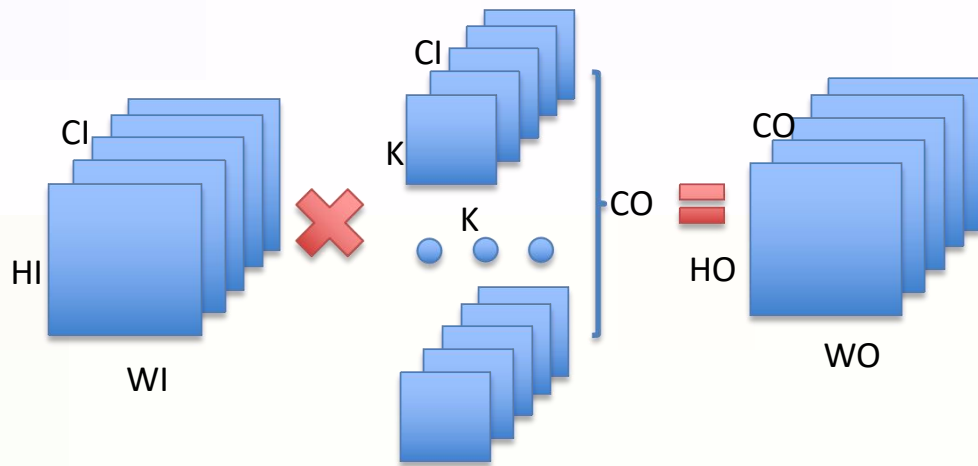


/Principle-parallelism

Spatial reuse: the same data is used by **more than one consumer** at **different spatial locations** of the hardware.



/Data reuse in DNN



Design Space

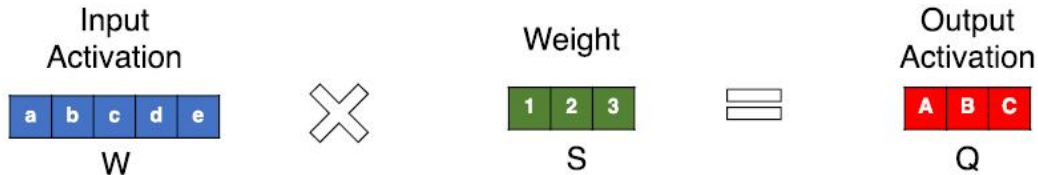
- Input activation
 - Size: $HI * WI * CI$
 - Reuse: $K * K * CO$
- Output activation
 - Size: $HO * WO * CO$
 - Reuse: $K * K * CI$
- Weight
 - Size: $K * K * CI * CO$
 - Reuse: $HO * WO$

/Dataflow

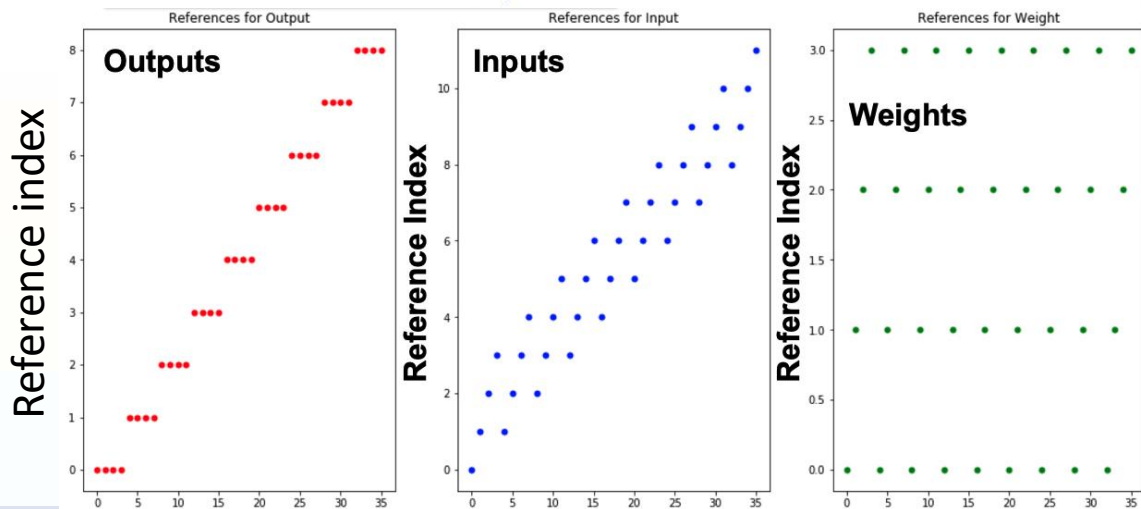
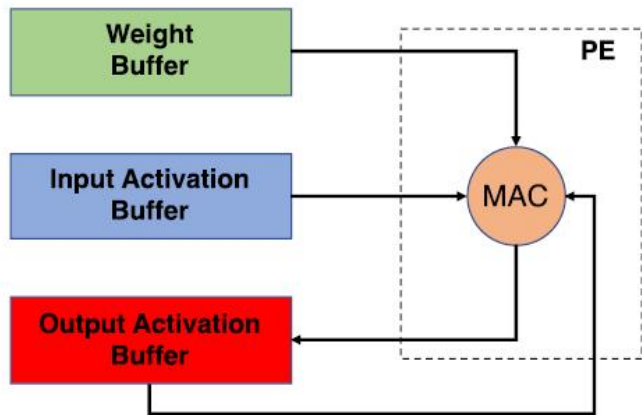
- Defines the execution order of the DNN operations in hardware.
 - Computation order
 - Data movement order
- Loop nest is a compact way to describe the execution order, i.e., dataflow, supported in hardware.
 - **for**: temporal for, describes the temporal execution order.
 - **spatial_for**: describes parallel execution.

/ Example

- No data reuse



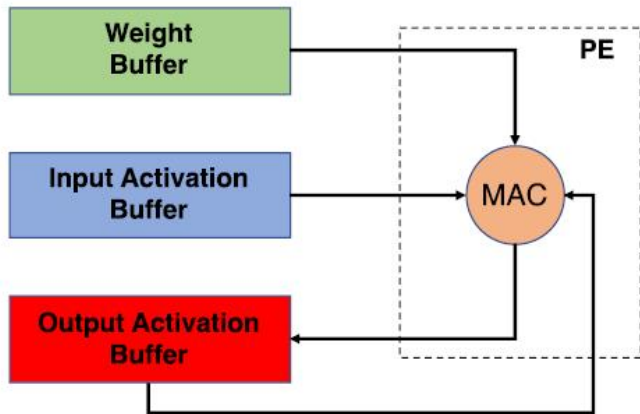
```
for (q=0; q<Q; q++){ // Q =9
    for (s=0; s<S; s++){ // S=4
        OA[q] += IA[q+s] * W[s];
    }
}
```



cycle

/ Example

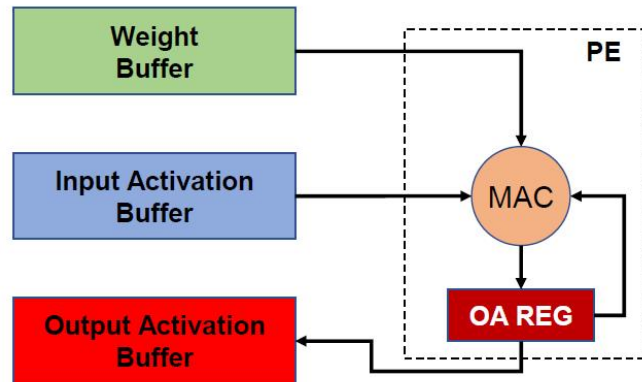
- Reuse for output activation



No data reuse

Output Activation
Q

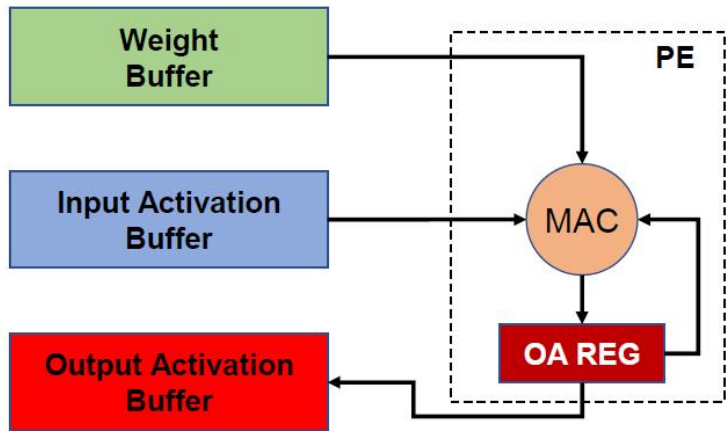
```
for (q=0; q<Q; q++) { // Q = 9
  for (s=0; s<S; s++) { // S=4
    OA[q] += IA[q+s] * W[s];
  }
}
```



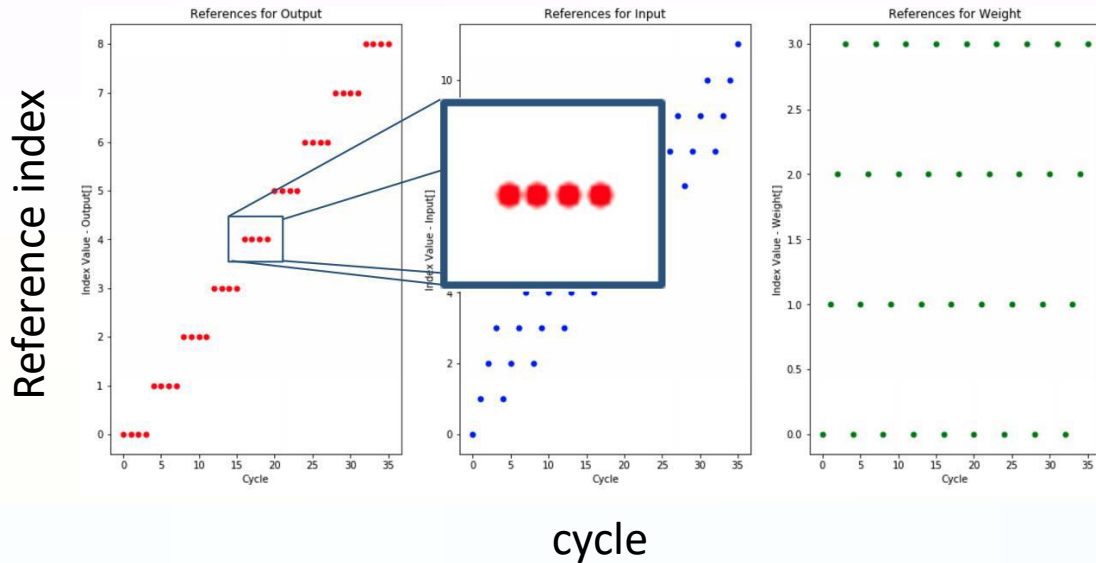
Reuse single outputs for S times.

/ Example

- Reuse for output activation

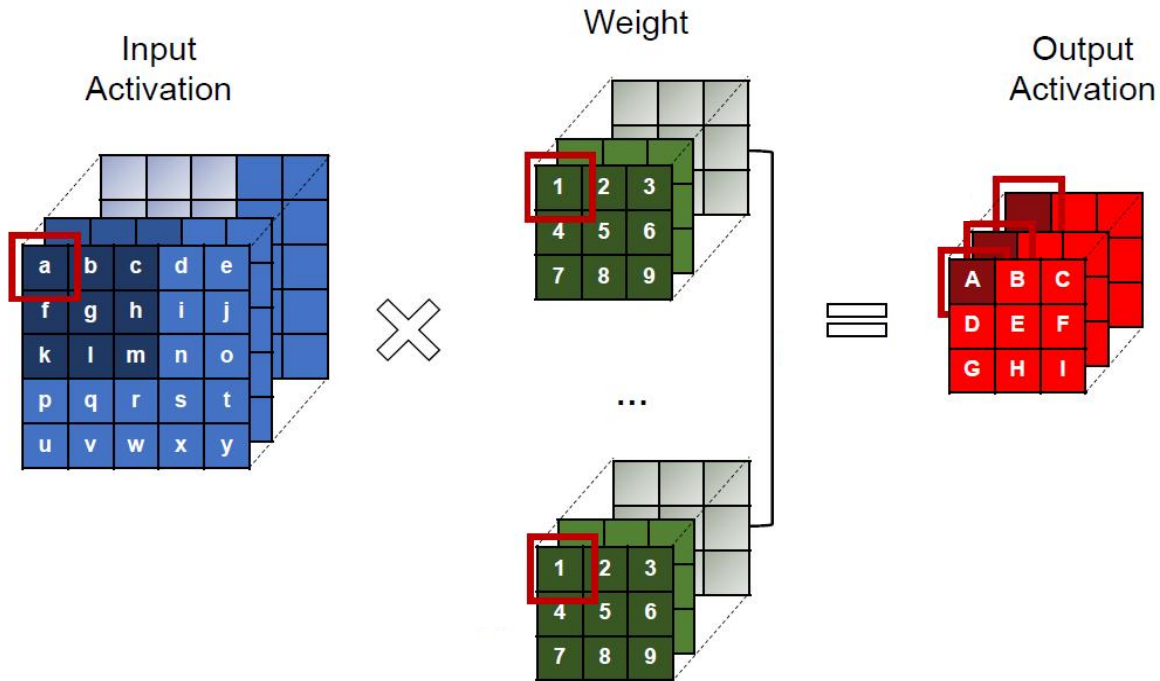


Reuse single outputs for S times.



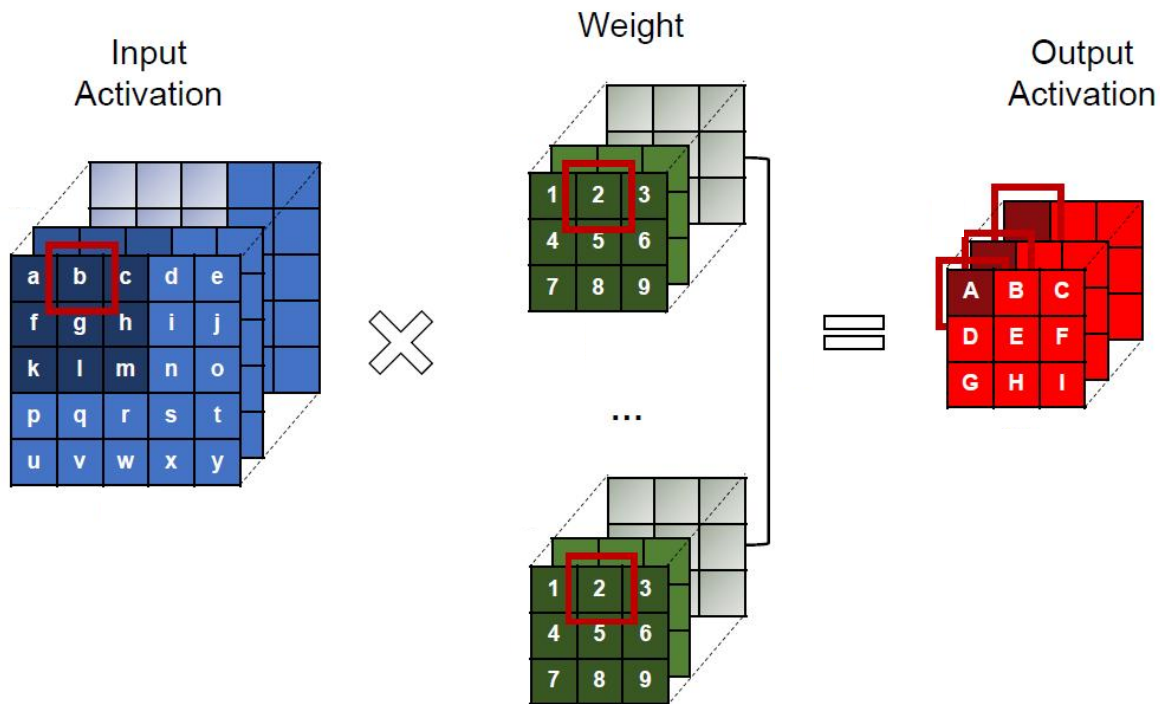
/ Example

- Reuse for output activation



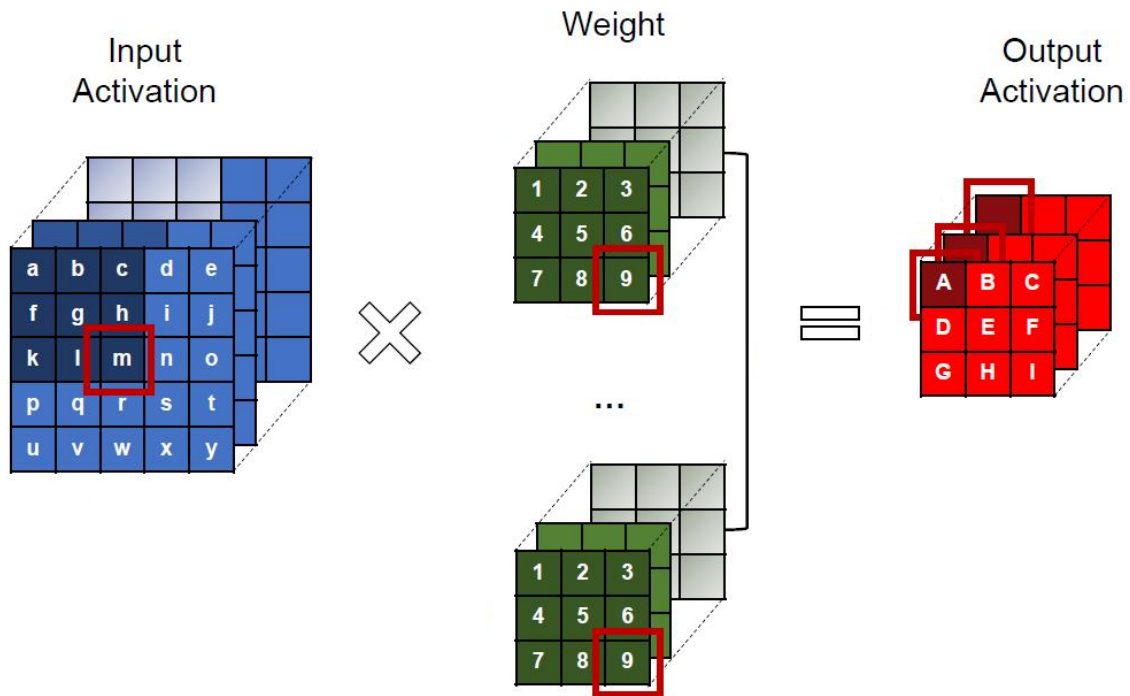
/ Example

- Reuse for output activation



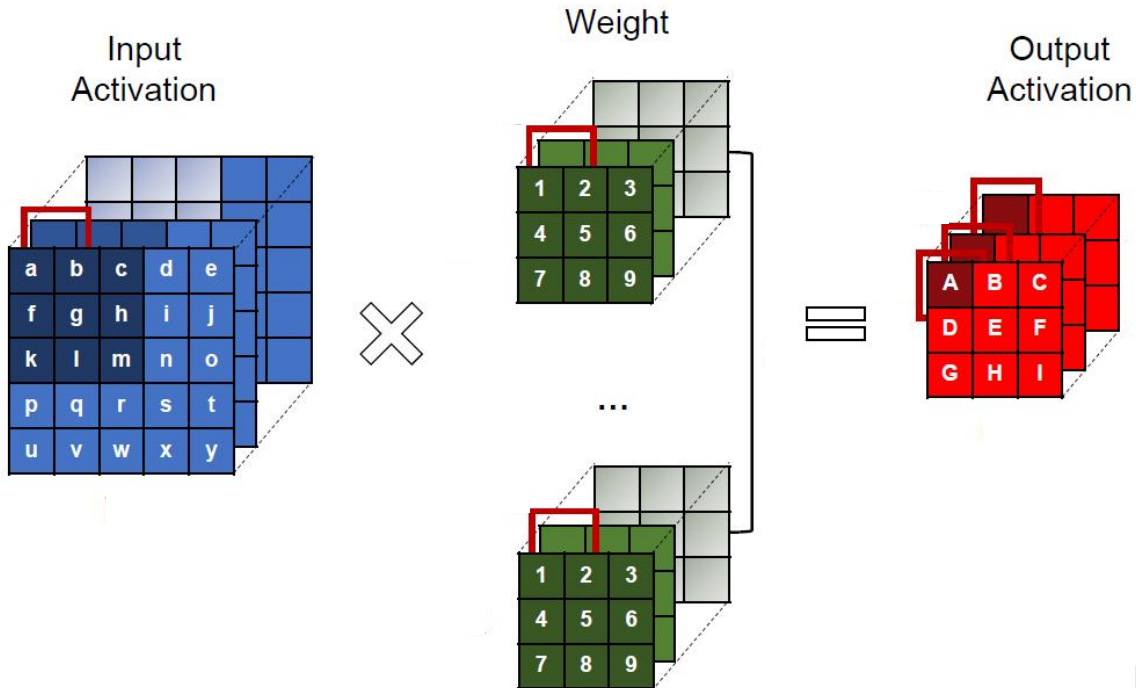
/ Example

- Reuse for output activation



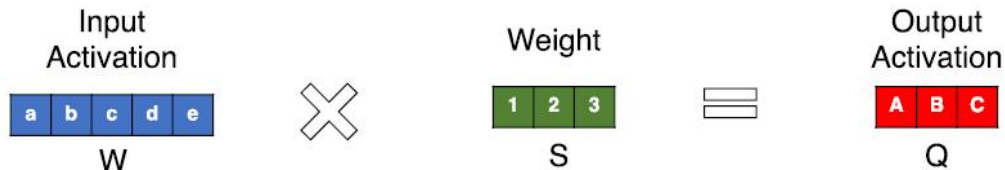
/ Example

- Reuse for output activation

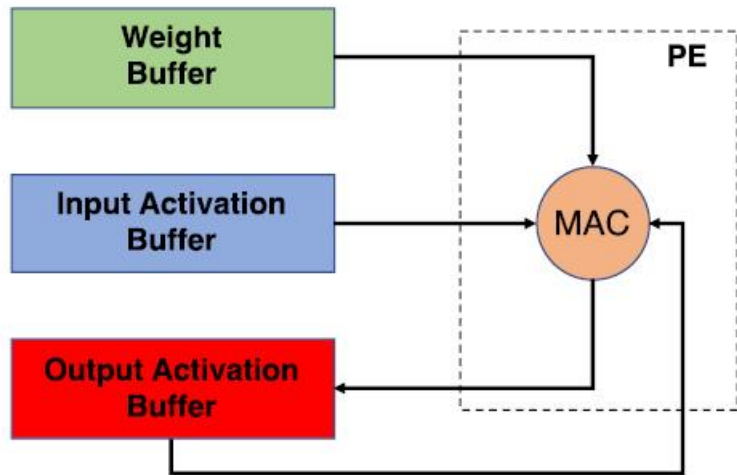


/ Example

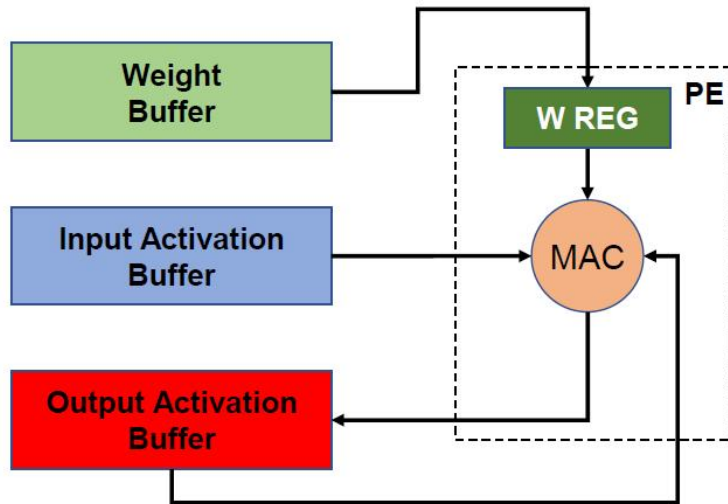
- Reuse for weights



```
for (s=0; s<S; s++) { // S=4
  for (q=0; q<Q; q++) { // Q=9
    OA[q] += IA[q+s] * W[s];
```



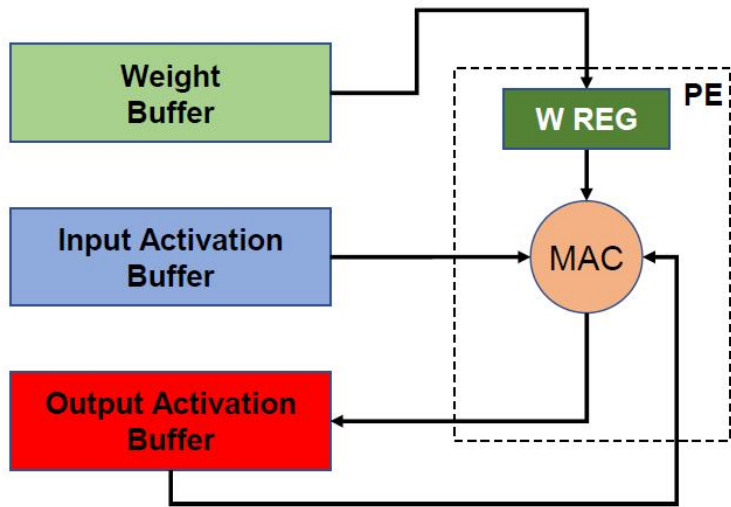
No data reuse



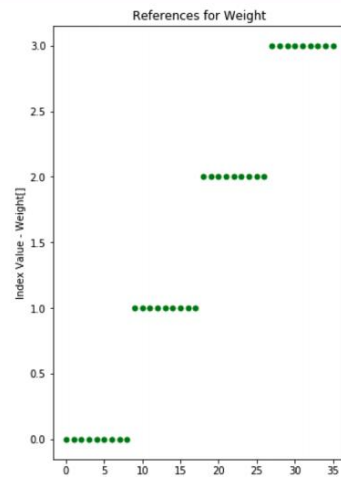
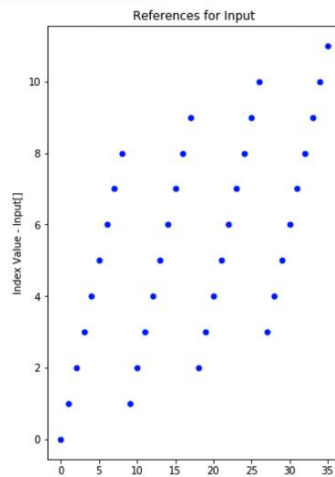
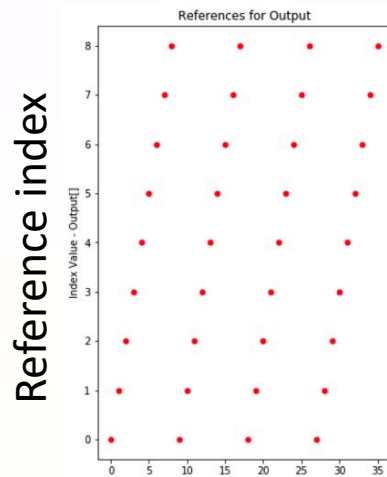
Reuse single weight for Q times.

/ Example

- Reuse for weights



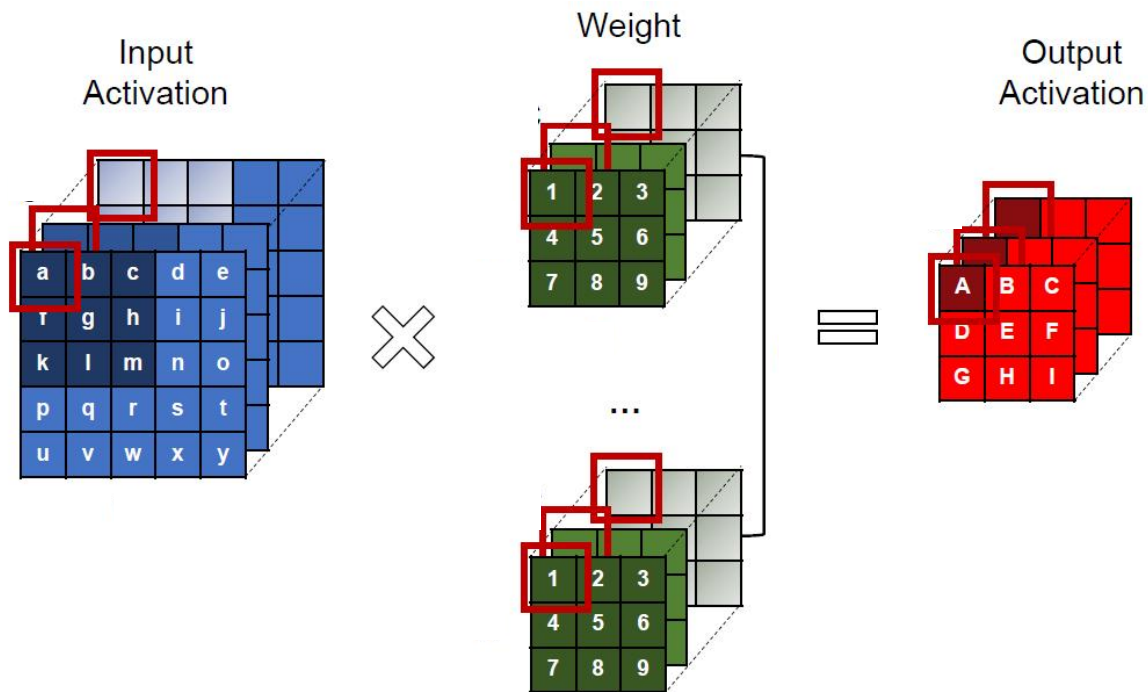
Reuse single weight for Q times.



cycle

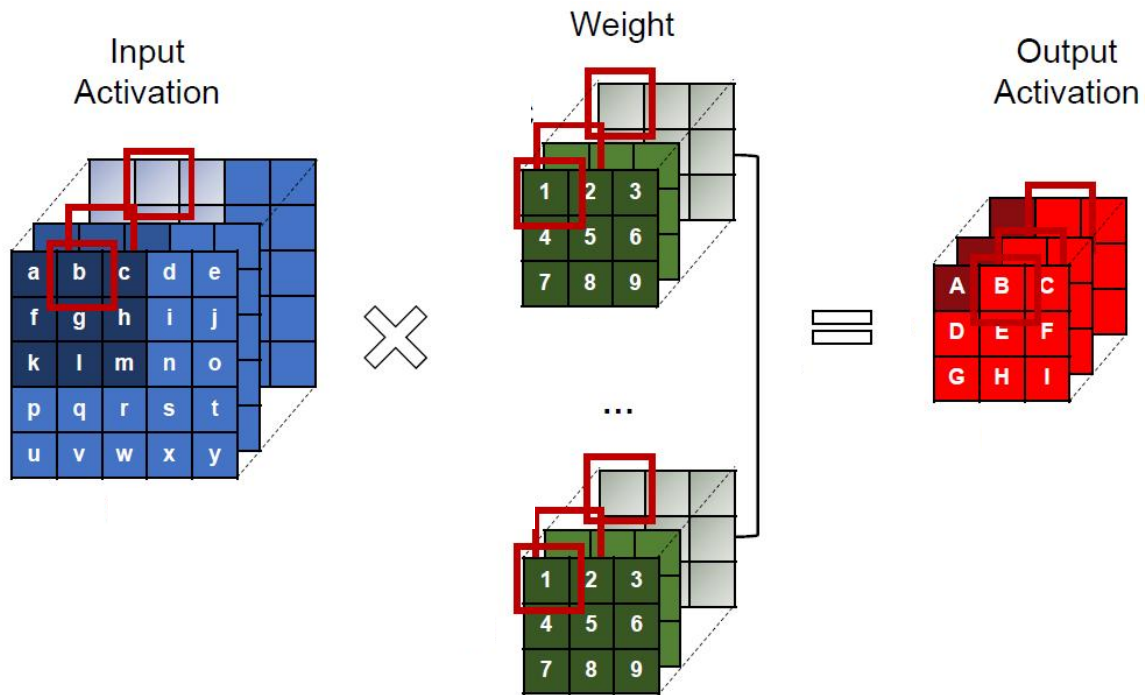
/ Example

- Reuse for weights



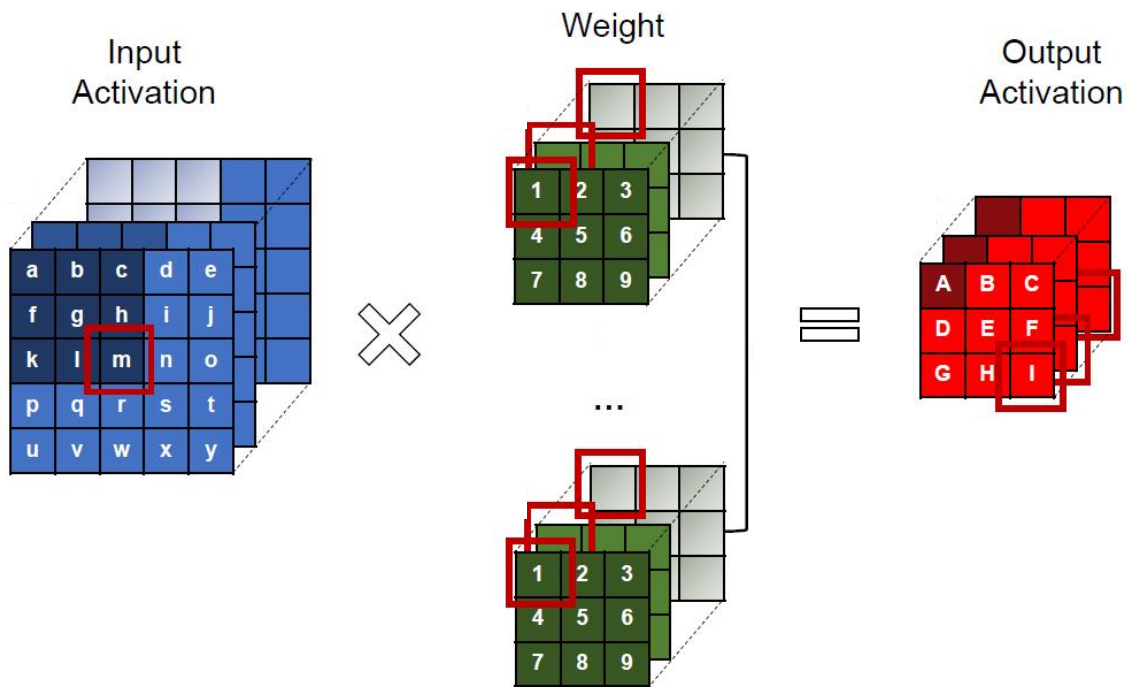
/ Example

- Reuse for weights



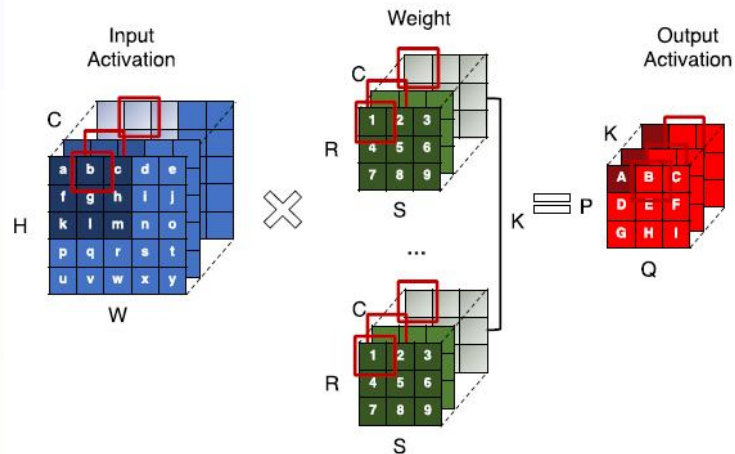
/ Example

- Reuse for weights



/ Research & project

- NVDLA by Nvidia



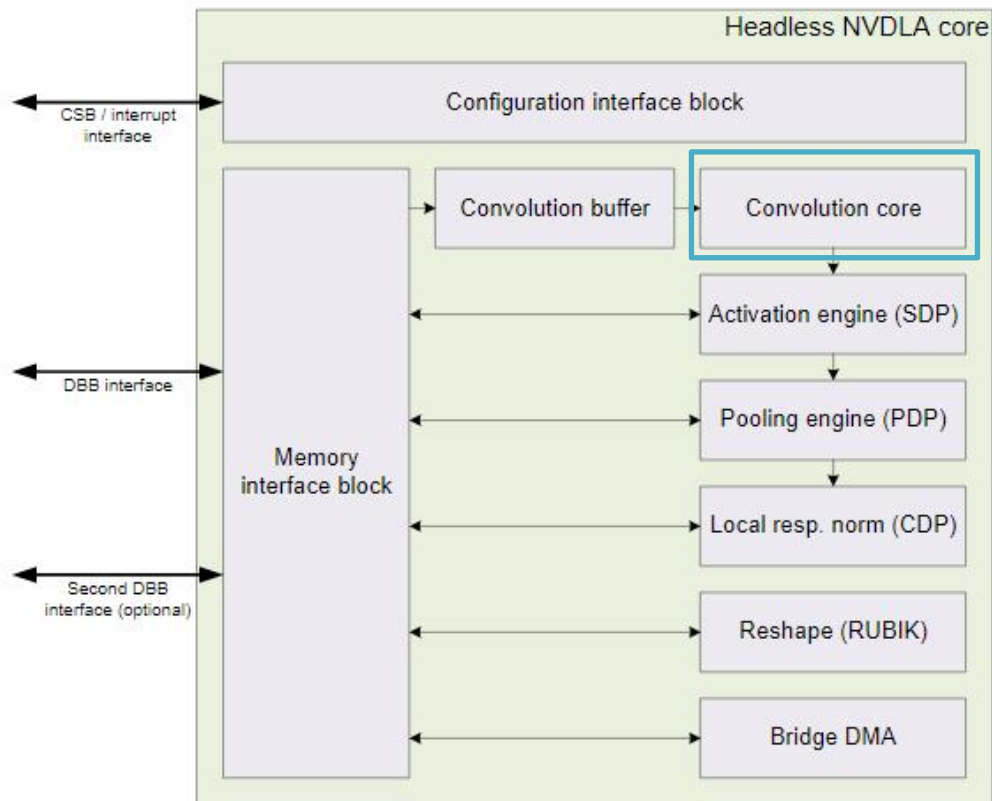
- **spatial_for**: describes parallel execution.

• NVDLA Dataflow (nvdla.org)

```
for (n=0; n<N; n++) {  
    for (r=0; r<R; r++) {  
        for (s=0; s<S; s++) {  
            for (c_t=0; c_t<C/16; c_t++) {  
                for (k_t=0; k_t<K/64; k_t++) {  
                    spatial_for (c_s=0; c_s<16; c_s++) {  
                        spatial_for (k_s=0; k_s<64; k_s++) {  
                            int curr_c = c_t * 16 + c_s;  
                            int curr_k = k_t * 64 + k_s;  
                            float curr_w = W[r][s][curr_c][curr_k];  
                            for (p=0; p<P; p++) {  
                                for (q=0; q<Q; q++) {  
                                    h = p * stride - pad + r;  
                                    w = q * stride - pad + s;  
                                    OA[n][curr_k][p][q] +=  
                                        IA[n][curr_c][h][w]  
                                        * curr_w;  
                                }  
                            }  
                        }  
                    }  
                }  
            }  
        }  
    }  
}
```

/ Research & project

- The architecture of NVDLA by Nvidia

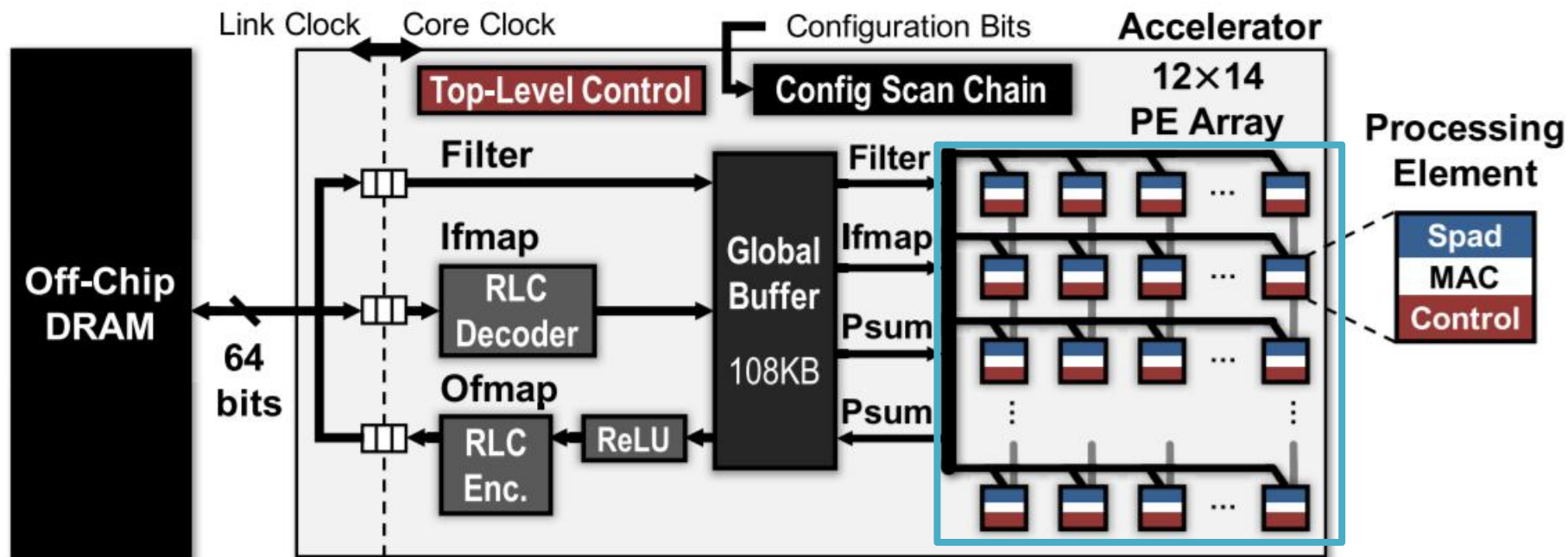


← What we focus in this lecture

/ Research & project

- Eyeriss

JSSC, 2017



/Research & project

- Eyeriss

JSSC, 2017

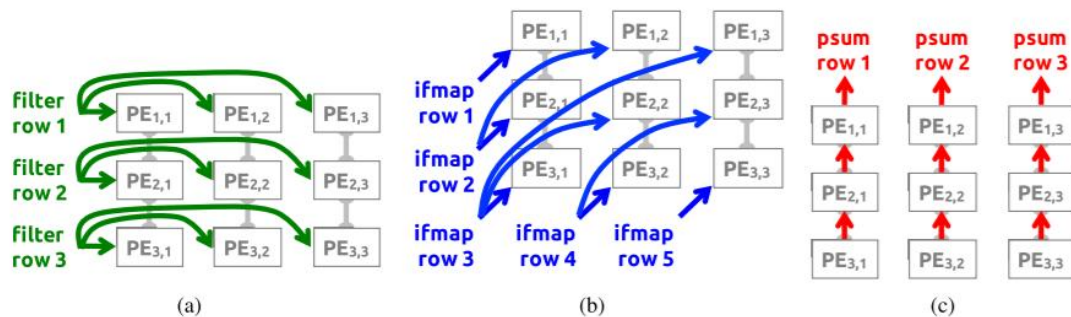


Fig. 4. Dataflow in a PE set for processing a 2-D convolution. (a) Rows of filter weights are reused across PEs horizontally. (b) Rows of ifmap values are reused across PEs diagonally. (c) Rows of psums are accumulated across PEs vertically. Reuse and accumulation of data within a PE set reduce accesses to the GLB and DRAM, saving data movement energy cost. In this example, the number of filter rows (R), ifmap rows (H), and ofmap rows (E) are 3, 5, and 3, respectively. Therefore, the PE set size is 3×3 . Filter and ifmap values from different rows are sent to the PE set in a time-interleaved fashion; all the PEs that reuse the same value receive it at the same cycle. The psums generated from one PE are sent to its neighbor PE immediately.

Three types of data reuse

/ Research & project

- TPU by Google

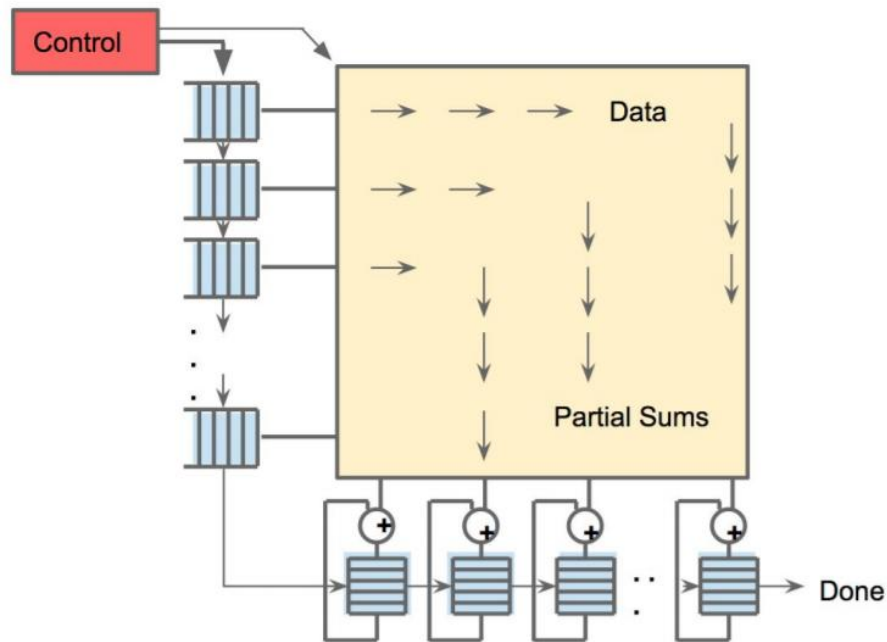
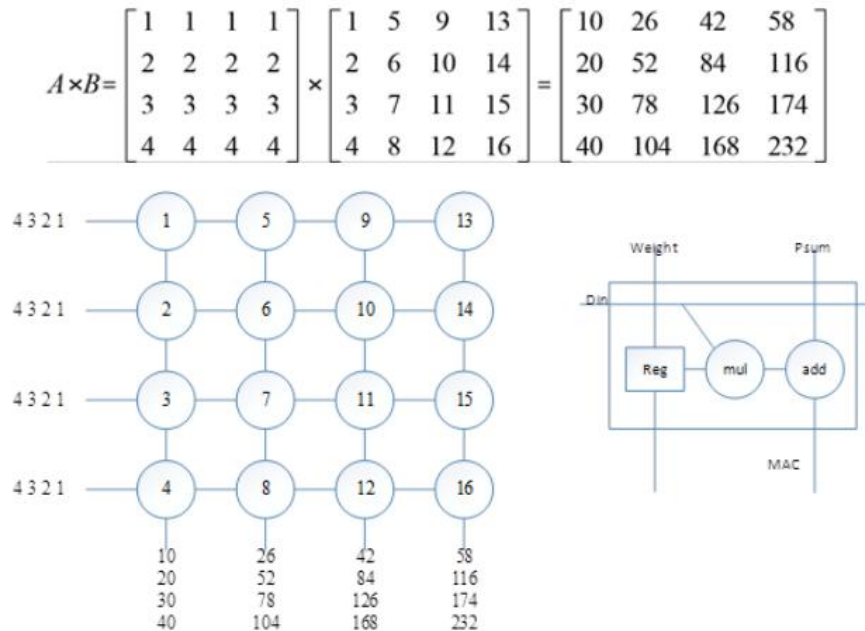


Figure 4. Systolic data flow of the Matrix Multiply Unit. Software has the illusion that each 256B input is read at once, and they instantly update one location of each of 256 accumulator RAMs.



脉动阵列

/ Research & project

- Google TPU

<i>Application</i>	<i>MLP0</i>	<i>MLP1</i>	<i>LSTM0</i>	<i>LSTM1</i>	<i>CNN0</i>	<i>CNN1</i>	<i>Mean</i>	<i>Row</i>
Array active cycles	12.7%	10.6%	8.2%	10.5%	78.2%	46.2%	28%	1
Useful MACs in 64K matrix (% peak)	12.5%	9.4%	8.2%	6.3%	78.2%	22.5%	23%	2
Unused MACs	0.3%	1.2%	0.0%	4.2%	0.0%	23.7%	5%	3
Weight stall cycles	53.9%	44.2%	58.1%	62.1%	0.0%	28.1%	43%	4
Weight shift cycles	15.9%	13.4%	15.8%	17.1%	0.0%	7.0%	12%	5
Non-matrix cycles	17.5%	31.9%	17.9%	10.3%	21.8%	18.7%	20%	6
RAW stalls	3.3%	8.4%	14.6%	10.6%	3.5%	22.8%	11%	7
Input data stalls	6.1%	8.8%	5.1%	2.4%	3.4%	0.6%	4%	8
TeraOps/sec (92 Peak)	12.3	9.7	3.7	2.8	86.0	14.1	21.4	9

Table 3. Factors limiting TPU performance of the NN workload based on hardware performance counters. Rows 1, 4, 5, and 6 total 100%

/Review

- **This lecture**
 - **Convolution**
 - **Basic convolution, depth-wise convolution, et al.**
 - **Implementation for convolution**
 - **Direct methods, GEMM, FFT-based, winograd**
 - **Pooling/BatchNorm**
 - **Merge the batchnorm into convolution layer**
 - **Computing Flow**
 - **Data reuse**
 - **Classic research & project**